NAAN MUDHALVAN • TN SKILLS ACADEMIC INITIATIVE

Script-Controlled Access Control List (ACL)

Restricting Record Access Based on Field Values in ServiceNow

Project Title:	**Script-Controlled ACL: Restrict Record Access**
Candidate Name:	**SHARAN CHAKKARAVARTHY M**
Program:	**Naan Mudhalvan / TN Skills Initiative**
Platform:	**ServiceNow Utah / Vancouver / Washington DC**
Update Set:	**ACL Project [Global]**
Custom Table:	**`u_institution_details` (Institution Details)**
GitHub Repository:	**GitHub Link**
Evaluation Date:	**October 2026**

Executive Summary

This project, titled **"Script-Controlled ACL: Restrict Record Access Based on Field Value"**, was engineered as a core requirement under the **Naan Mudhalvan / TN Skills** technical upskilling initiative. The primary goal of this project is to implement granular, dynamic data segregation within an enterprise ServiceNow instance using **Access Control Lists (ACLs)** powered by server-side JavaScript conditions.

In enterprise institutions managing multifaceted student records across diverse engineering disciplines (such as Electrical and Electronics Engineering [EEE], Computer Science and Engineering [CSE], and Electronics and Communication Engineering [ECE]), ensuring data

compartmentalization is vital. Traditional role-based access allows all-or-nothing visibility. By contrast, a **script-controlled ACL** evaluates both the user's assigned role and the specific record's field attributes (e.g., `Branch == 'EEE'`) at runtime, implementing the **Principle of Least Privilege (PoLP)** and guaranteeing strict confidentiality.

Key Architectural Highlights

- **Granular Role Segmentation:** Segregation of responsibilities into distinct custom security roles (`bb1` for Read, `bb2` for Create, `bb3` for Write, and `bb4` for Delete).
- **Runtime Script Logic:** Implementation of server-side GlideSystem (`gs`) JavaScript to dynamically grant full access to administrators while confining departmental operators exclusively to their branch records.
- **Full CRUD Lifecycle Enforcement:** Comprehensive configuration and impersonation verification across Create, Read, Update/Write, and Delete operations on the `u_institution_details` table.

Project Architecture & Security Evaluation Hierarchy

ServiceNow enforces access control through a rigorous 3-tiered evaluation pipeline on every transaction. An incoming request must satisfy all three criteria sequentially:

1. **Roles:** Does the requesting user possess one of the roles configured in the `Requires role` related list?
2. **Conditions:** Does the targeted record satisfy the declarative conditions defined in the Condition Builder (e.g., `Branch = EEE`)?
3. **Script:** Does the advanced JavaScript snippet return `true` (or evaluate `answer = true`)?

If any of these stages fail, ServiceNow immediately rejects access. Furthermore, Table-level ACLs (e.g., `u_institution_details`) are evaluated before Field-level ACLs (e.g., `u_institution_details.*`), guaranteeing multi-layer defense.

Milestone	Task Name	Key Deliverable	Status
Milestone 1	Users and Roles Creation	Test User `EEEUser`, Custom Roles `bb1`, `bb2`, `bb3`, `bb4`	Completed
Milestone	Custom Table Creation	Table `u_institution_details` with 7	Completed

Milestone	Task Name	Key Deliverable	Status
2		Schema Columns & Records	
Milestone 3	Access Control List - READ	Elevate `security_admin`, Read ACL with Script, Multi-User Verification	Completed
Milestone 4	Access Control List - CREATE	Create ACL with Role `bb2`, New Button Validation	Completed
Milestone 5	Access Control List - WRITE	Write ACL with Role `bb3`, Inline & Form Editing Verification	Completed
Milestone 6	Access Control List - DELETE	Delete ACL with Role `bb4`, Record Deletion Confirmation Modal	Completed
Milestone 7	Conclusion & Documentation	Full Project Synthesis, Verification Matrix, Deliverables	Completed

Milestone 1: Creation of Users and Roles

The objective of Milestone 1 is to establish the identity and access management foundation. A dedicated test user representing the EEE departmental staff (`EEEUser`) was created along with four discrete custom security roles: `bb1` (Read), `bb2` (Create), `bb3` (Write), and `bb4` (Delete).

Step 1.1: Navigating to User Administration

From the ServiceNow Application Navigator, search for `users` under **System Security > Users and Groups > Users**. Click on **New** to open the user creation form.

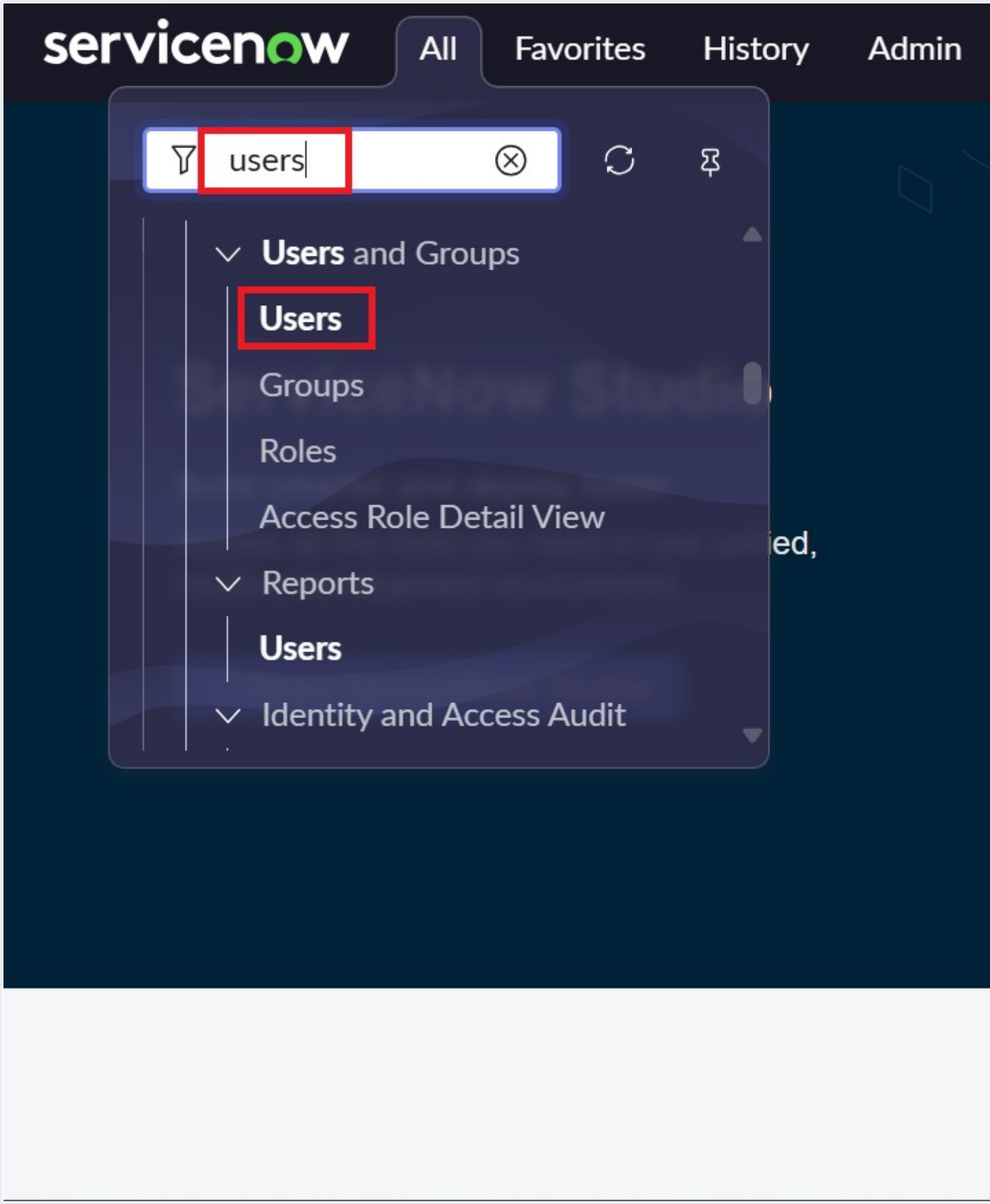


Figure 1: Navigating to User Administration > Users via Application Navigator

Step 1.2: Creating Test User (EEEUser)

Populate the user record with the following parameters and click **Update / Submit**:

- **User ID:** `EEEUser`
- **First name:** `EEE`
- **Last name:** `User`
- **Email:** `eeeuser@gmail.com`
- **Active:** `true` (Checked)

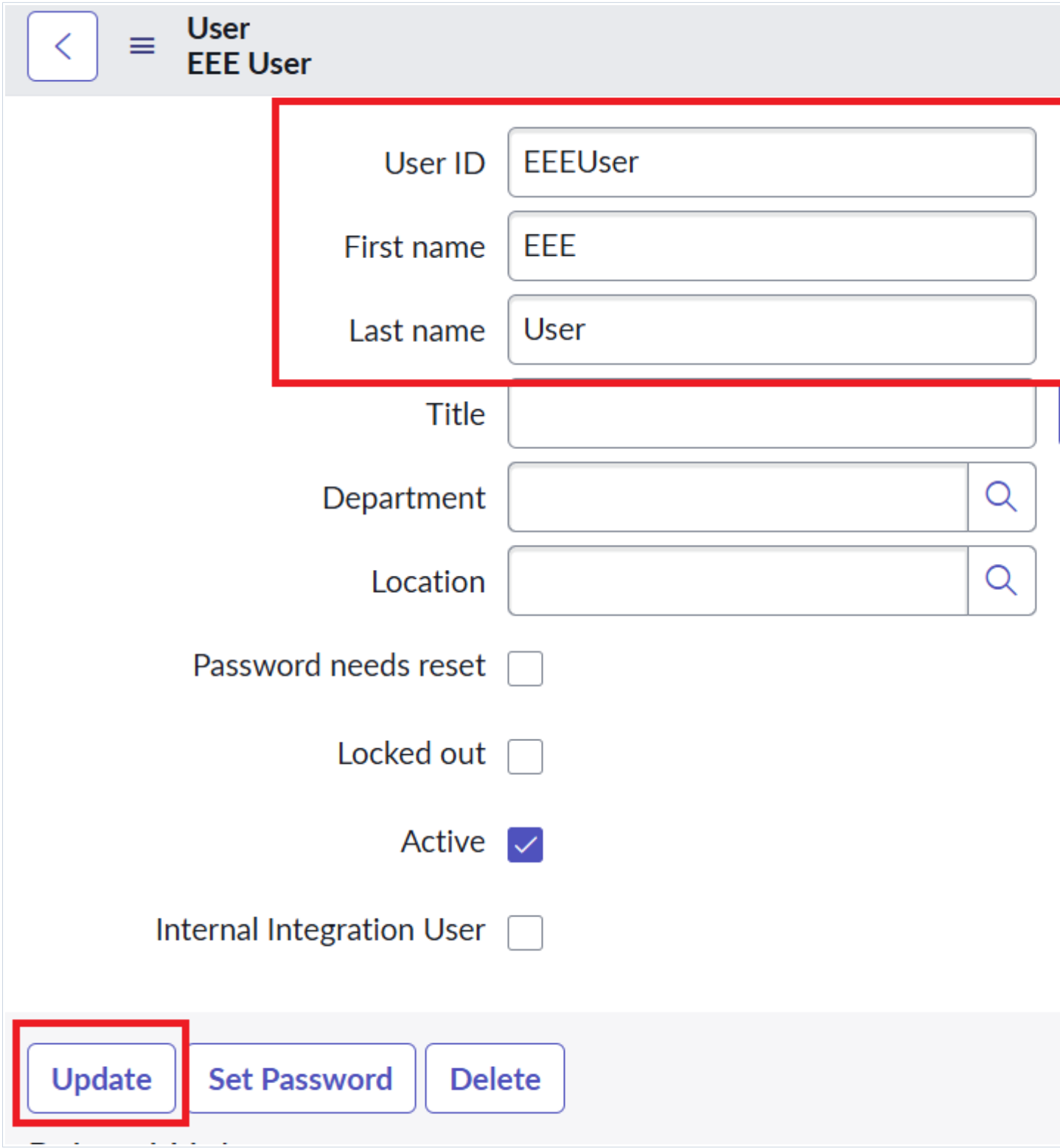


Figure 2: Creating Test User 'EEEUser' in sys_user Table

Step 1.3: Navigating to Roles Management

Search for `roles` in the Application Navigator under **System Security > Users and Groups > Roles**.

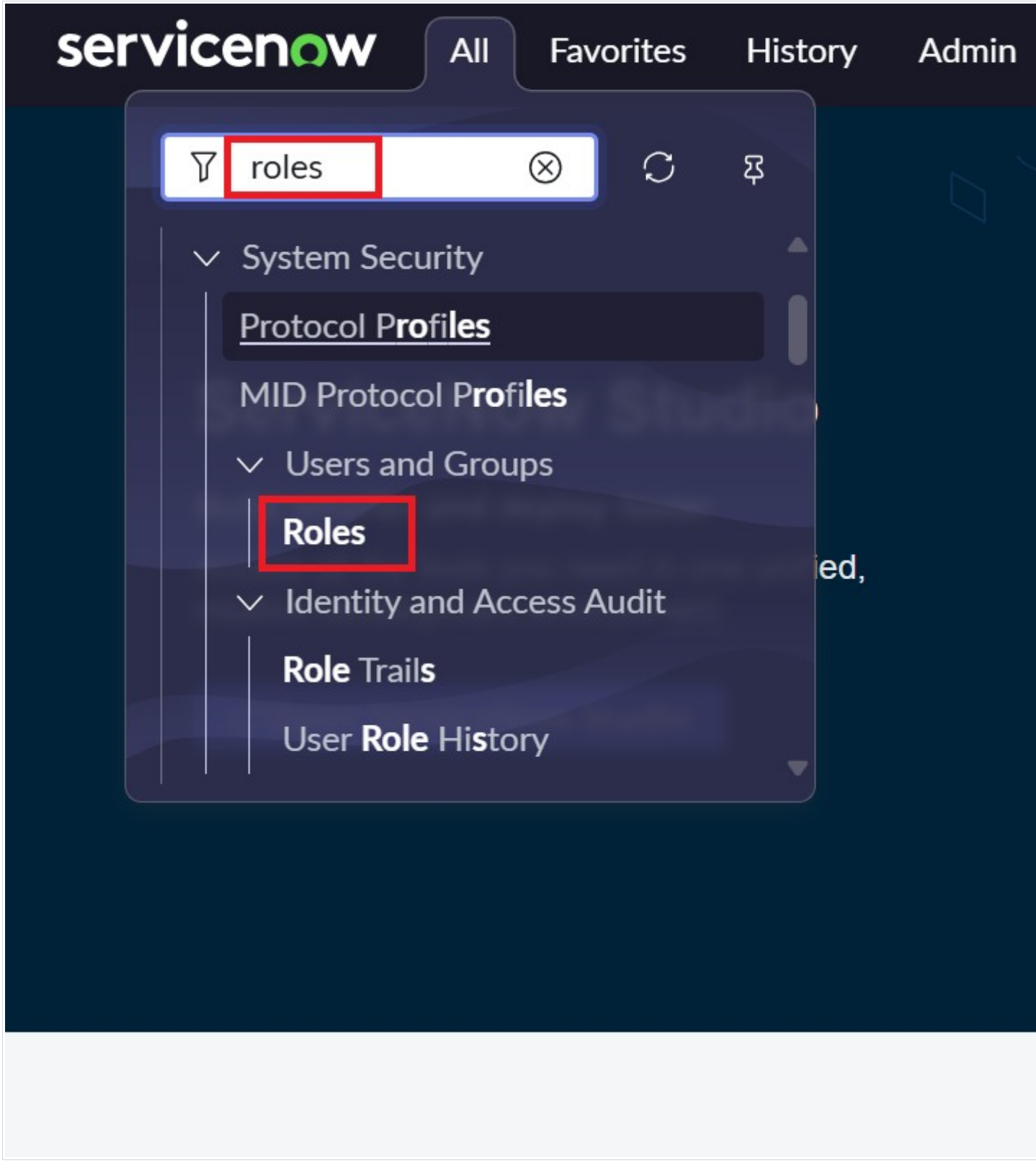


Figure 3: Accessing User Administration > Roles Interface

Step 1.4: Creating Custom Security Roles

Click **New** and define custom role `bb1`. Repeat the identical procedure to create roles `bb2`, `bb3`,

and bb4. Once created, associate all four roles with EEEUser to prepare for staged access verification.

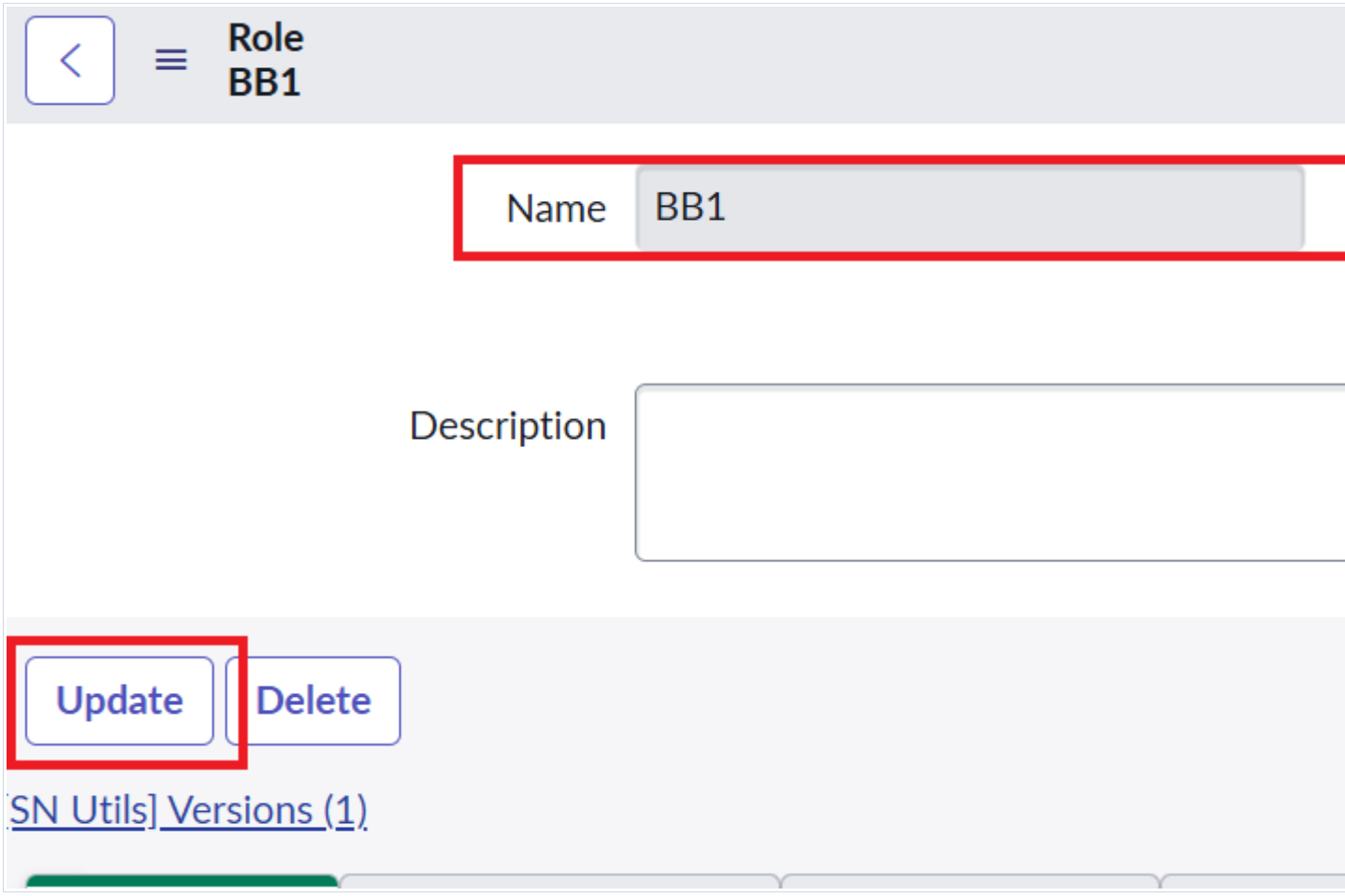


Figure 4: Creating Custom Role 'BB1' for Dynamic Access Control

Role Name	Application Scope	Designated Operation	Assigned To
bb1	Global	**READ** Access on EEE Branch Records	EEEUser
bb2	Global	**CREATE** Access on Table	EEEUser
bb3	Global	**WRITE / EDIT** Access on EEE Records	EEEUser
bb4	Global	**DELETE** Access on EEE Records	EEEUser

Milestone 2: Custom Table Creation (Institution Details)

Milestone 2 involves configuring the institutional data repository. A custom table named `u_institution_details` (Label: *Institution Details*) was created with seven specific fields and populated with test data across various departments to serve as the benchmark for access restriction tests.

Step 2.1: Accessing System Definition Tables

Navigate to **System Definition > Tables** via the Application Navigator.

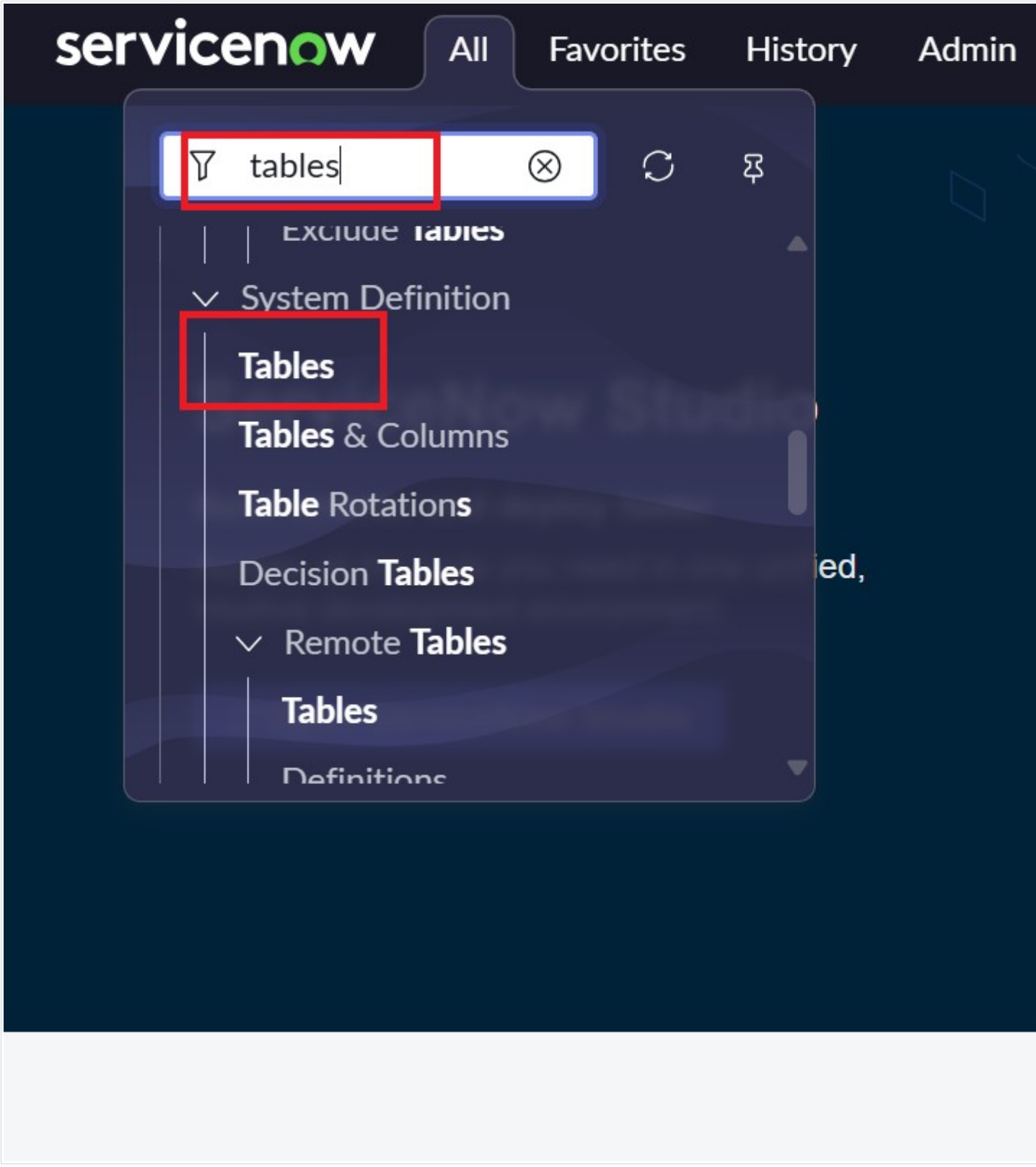


Figure 5: Navigating to System Definition > Tables

Step 2.2: Defining Custom Table Properties

Click **New** and specify table attributes:

- **Label:** `Institution Details`
- **Name:** `u_institution_details` (System generated prefix)
- **Extends table:** `-- None --` (Standalone table)
- **Application:** `Global`

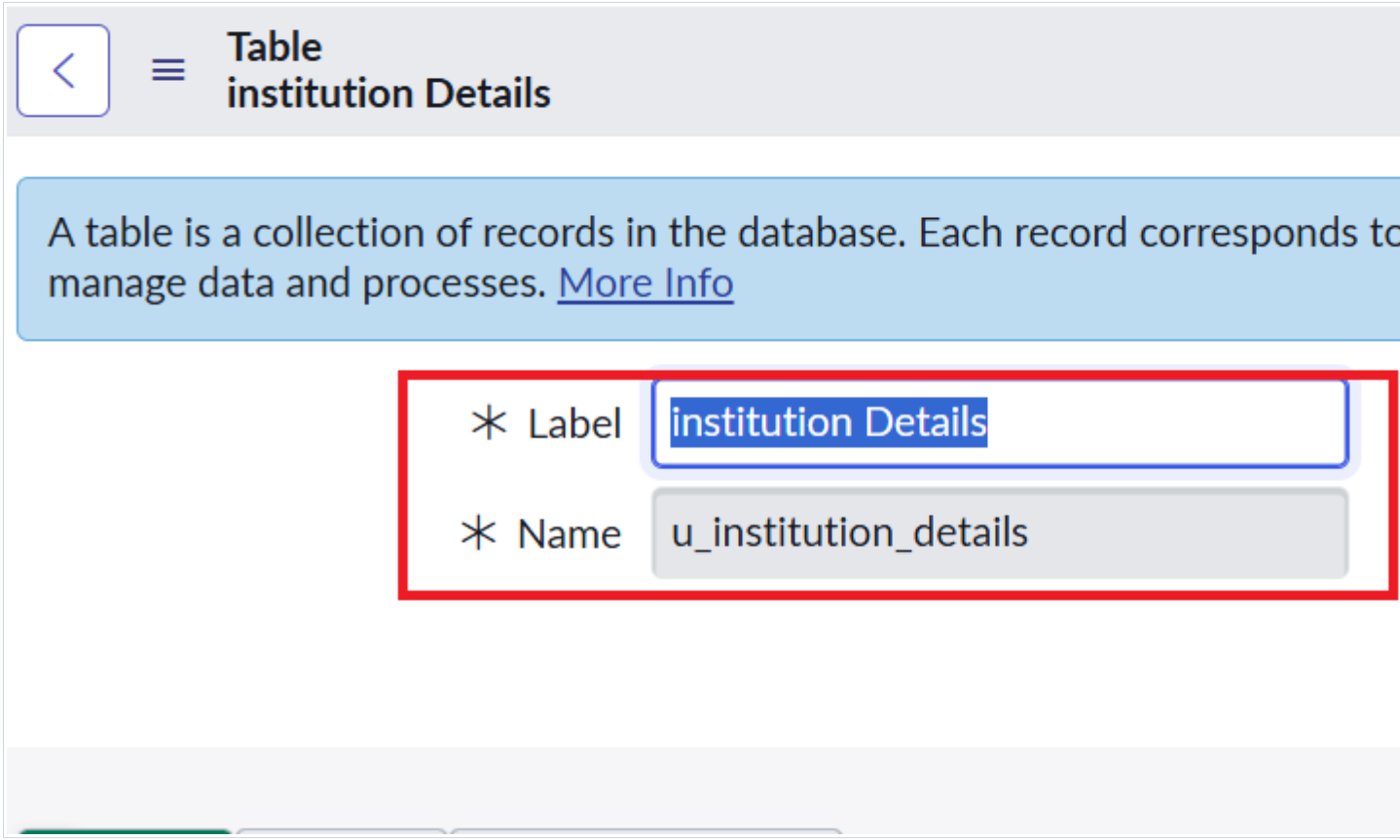


Figure 6: Custom Table Configuration for Institution Details

Step 2.3: Data Dictionary Columns Configuration

In the **Columns** embedded list, use the *Insert a new row* mechanism to establish the required data fields as specified in the Naan Mudhalvan curriculum:

Column Label	Column Name	Data Type	Reference / Details
Student Roll Number	`u_student_roll_number`	Auto Number	Prefix: INS (e.g., INS0001001)
Student Name	`u_student_name`	Reference	Referenced Table: User (`sys_user`)

Column Label	Column Name	Data Type	Reference / Details
Faculty Name	u_faculty_name	Reference	Referenced Table: User (sys_user)
Branch	u_branch	Choice	Options: ECE, EEE, CSE
Email	u_email	String	Max Length: 100
Phone Number	u_phone_number	String	Max Length: 40
Description	u_description	Multi-line String	Max Length: 4000

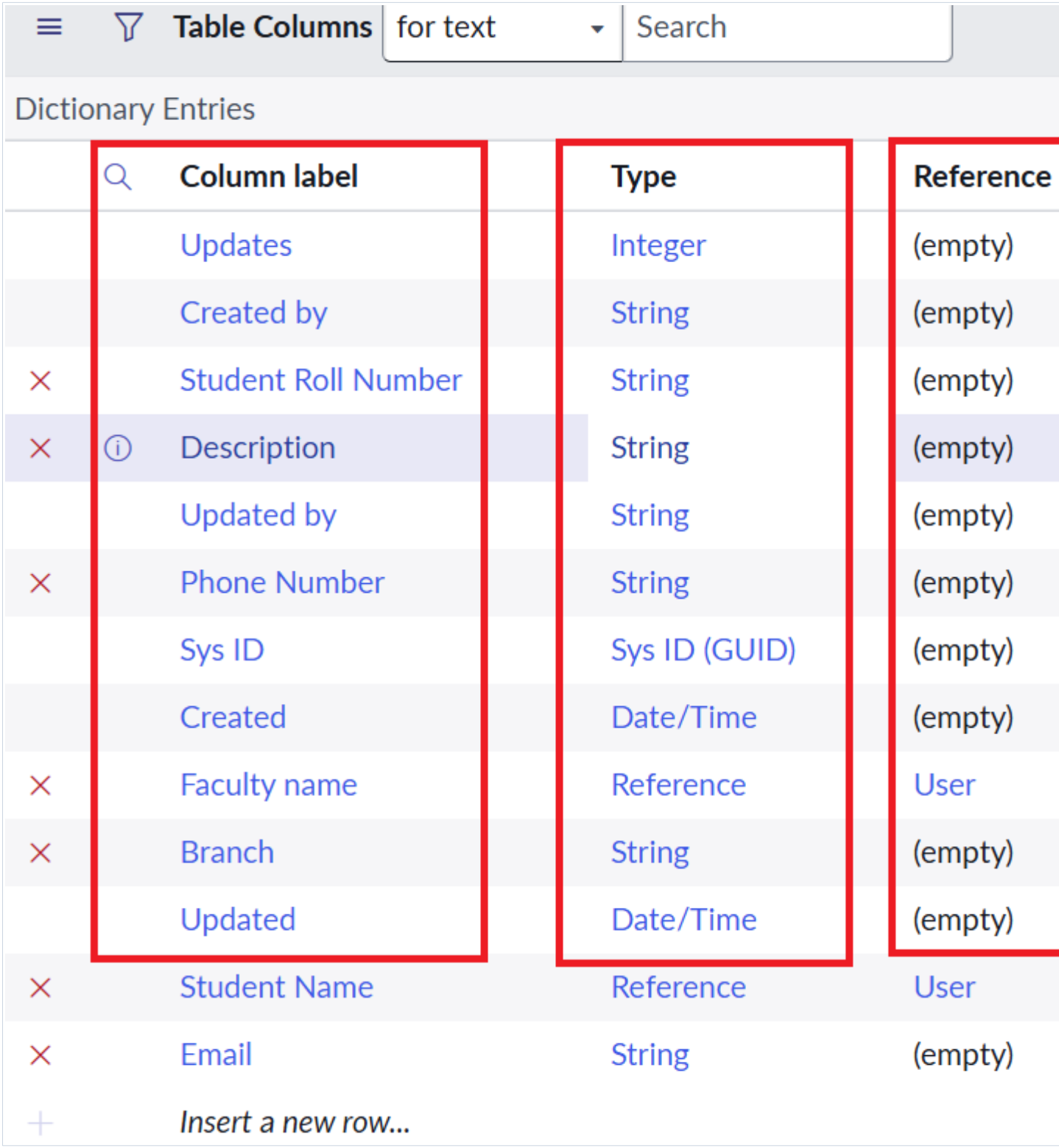


Figure 7: Data Dictionary Columns Schema for u_institution_details

Step 2.4: Populating Cross-Departmental Test Records

Under Related Links, click **Show List** (or enter `u_institution_details.list` in the navigator). Populate student records spanning **ECE**, **EEE**, and **CSE** branches to ensure all branches are

represented for filtering verification.

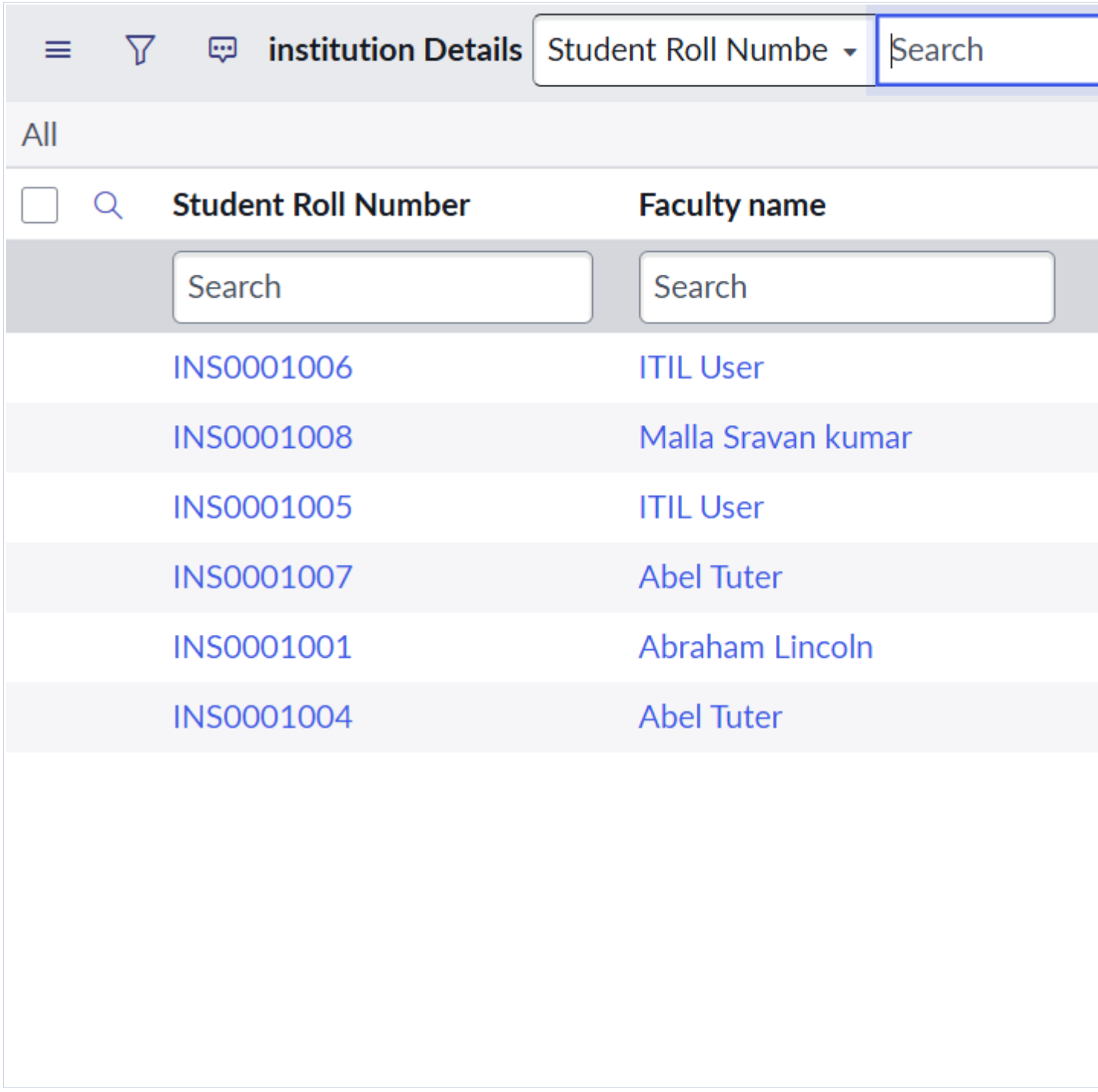


Figure 8: Baseline Multi-Department Records in Institution Details Table

Milestone 3: Access Control List - READ Configuration

Milestone 3 constitutes the core objective: restricting read visibility so that users holding the `bb1` role can **strictly view records belonging to the EEE branch**, unauthorized users are denied completely, and system administrators maintain global oversight.

Step 3.1: Server-Side Advanced Script Implementation

To evaluate field-level runtime conditions, an advanced JavaScript script was engineered for the Read ACL:

```
/** * ServiceNow Advanced Access Control Script * Table: u_institution_details | Operation: Read */ (function () { // Allow admin users full access if (gs.hasRole('admin')) { return true; } // Allow only EEE branch users to see EEE records if (gs.hasRole('bb1')) { return true; } // Deny access for all others return false; })();
```

```
(function () {
    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }
    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1')){
        return true;
    }
    // Deny access for all others
    return false;
})();
```

Figure 9: Advanced JavaScript Logic for Script-Controlled Read Access

Step 3.2: Elevating Privileges to Security Admin

Creating and modifying Access Control Rules requires elevated security privileges. In ServiceNow, navigate to the user profile menu and activate the **security_admin** elevated role.

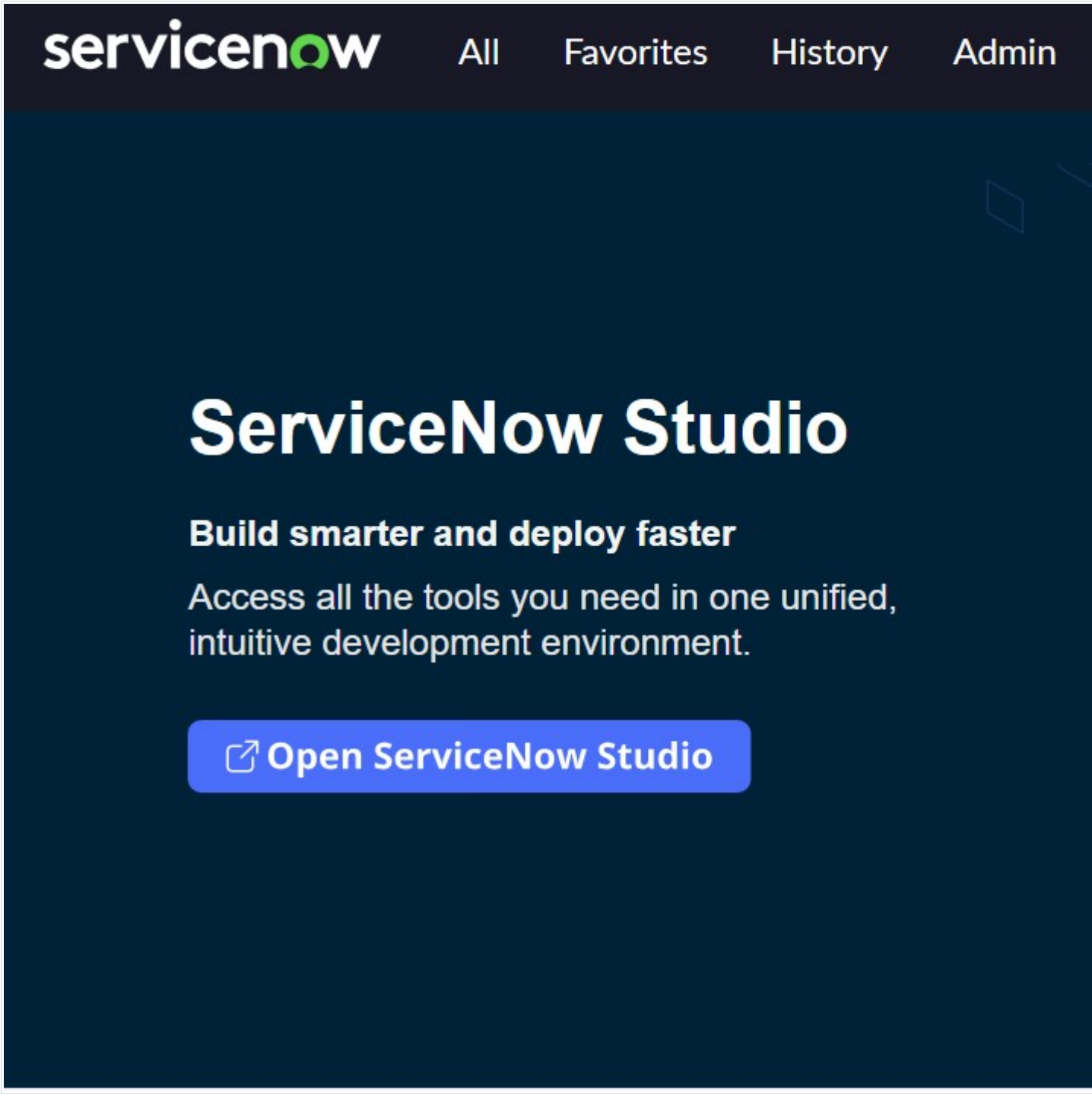


Figure 10: Navigating to System Security > Access Control (ACL)

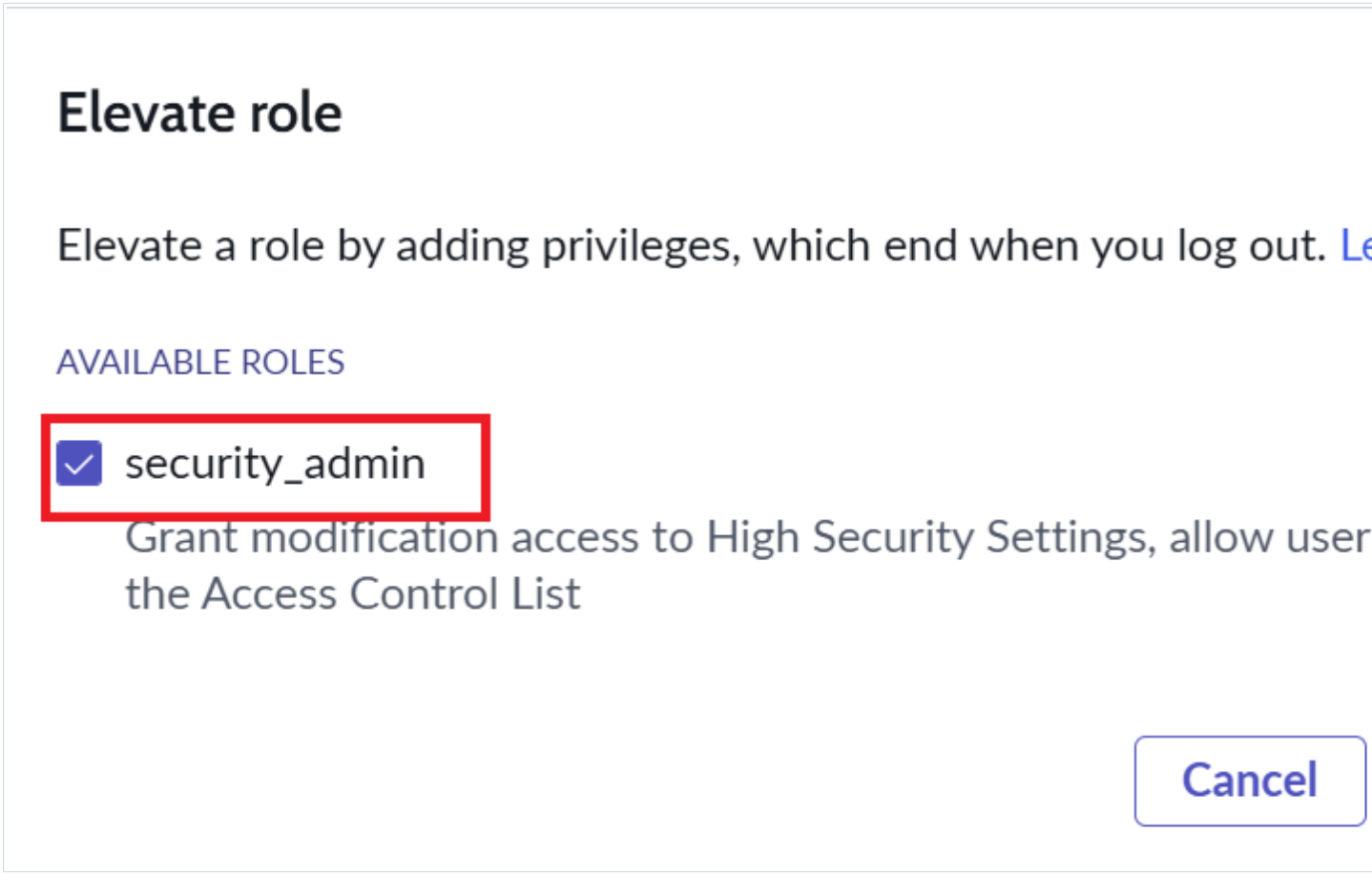


Figure 11: Elevating Privileges to security_admin Role

Step 3.3: Configuring the Read Access Control Rule

Navigate to **System Security > Access Control (ACL)** and click **New**:

- **Type:** `record`
- **Operation:** `read`
- **Name:** `u_institution_details` (Table level)
- **Active:** `true` (Checked)
- **Advanced:** `true` (Checked)
- **Requires role:** `bb1`
- **Condition:** `Branch [u_branch] is EEE`
- **Script:** Embed the script developed in Step 3.1

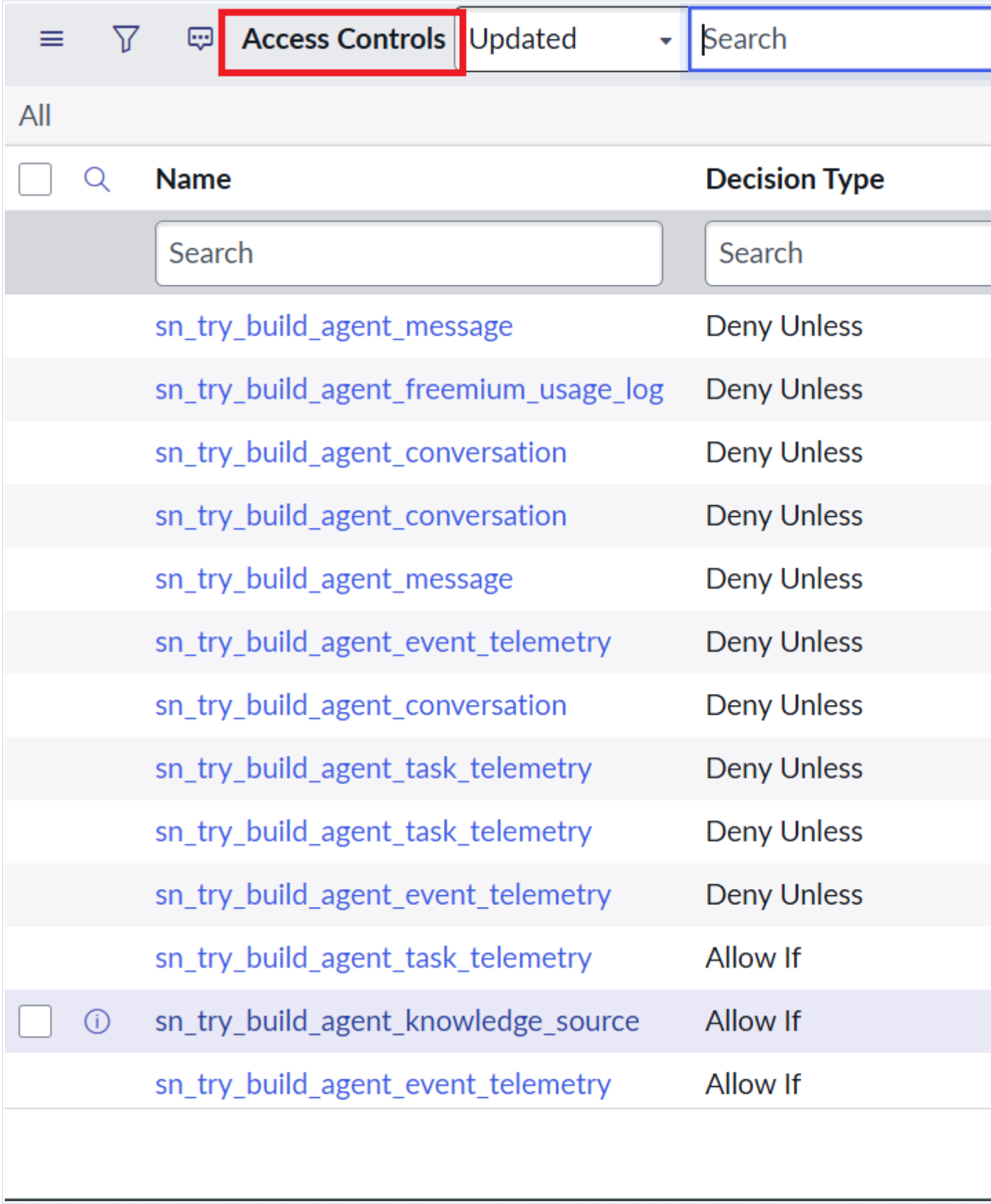


Figure 12: Access Control List Registry (sys_security_acl)

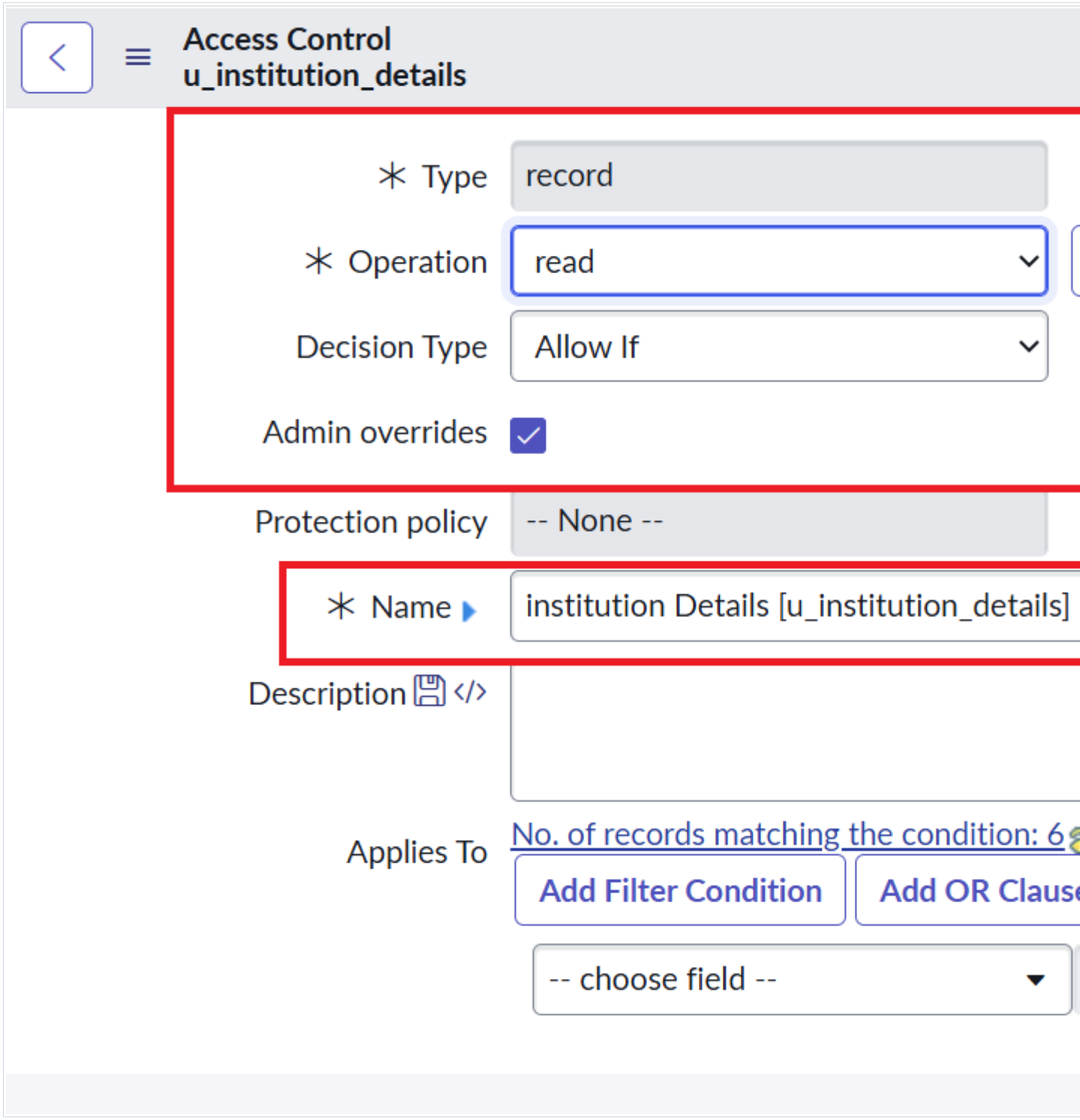


Figure 13: Read Access Control Rule Definition

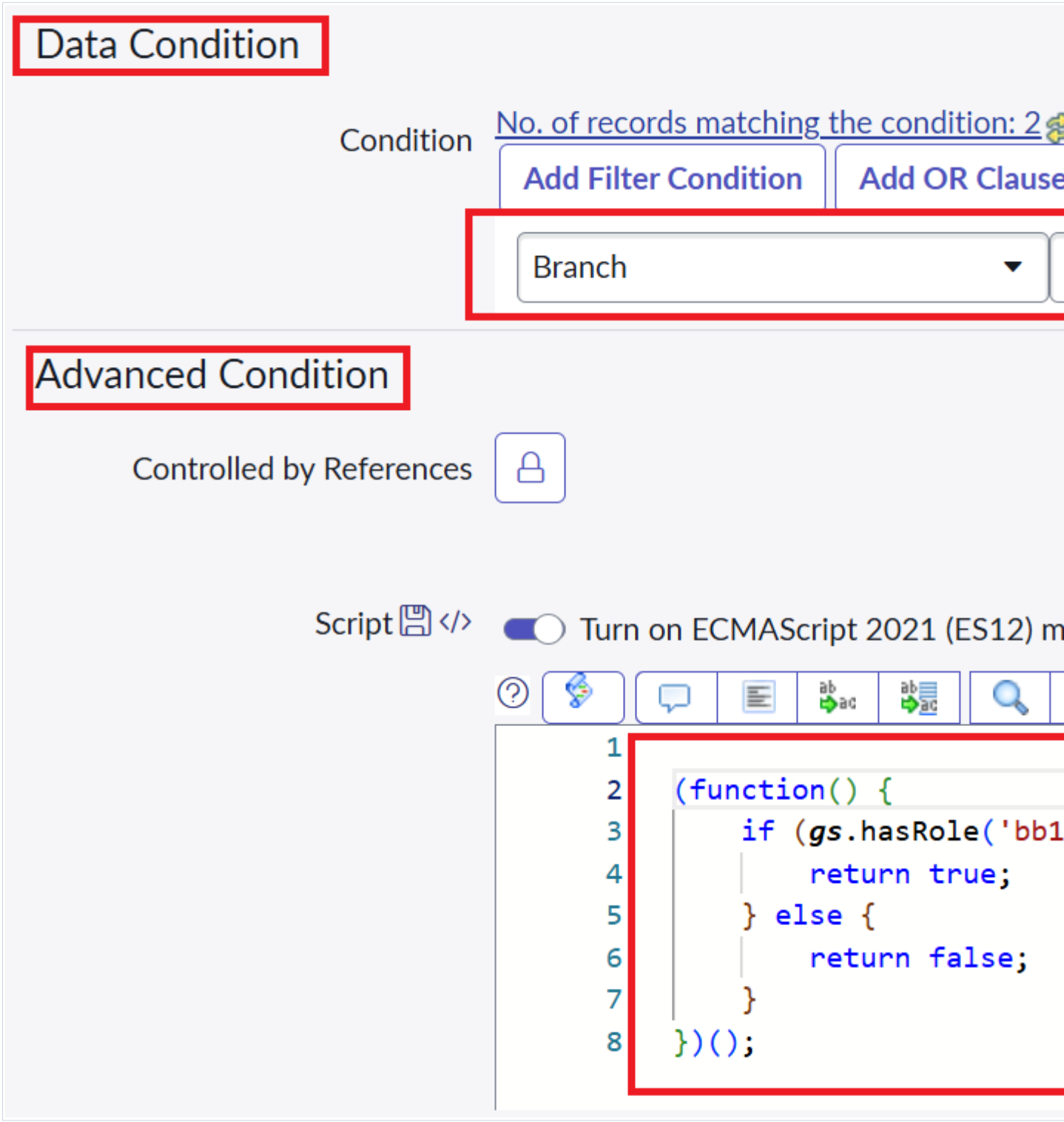


Figure 14: Configuring Branch is EEE Condition and Advanced Script

Step 3.4: Rigorous Multi-Persona Verification (READ)

To confirm compliance with project requirements, three distinct impersonation tests were conducted:

Test 1: Impersonate EEEUser (Possessing bb1 role)

When `EEEUser` accesses `u_institution_details.list`, only records matching `Branch == EEE` (INS0001007 and INS0001004) are rendered. Non-EEE records (CSE, ECE) are completely hidden at the database query level.

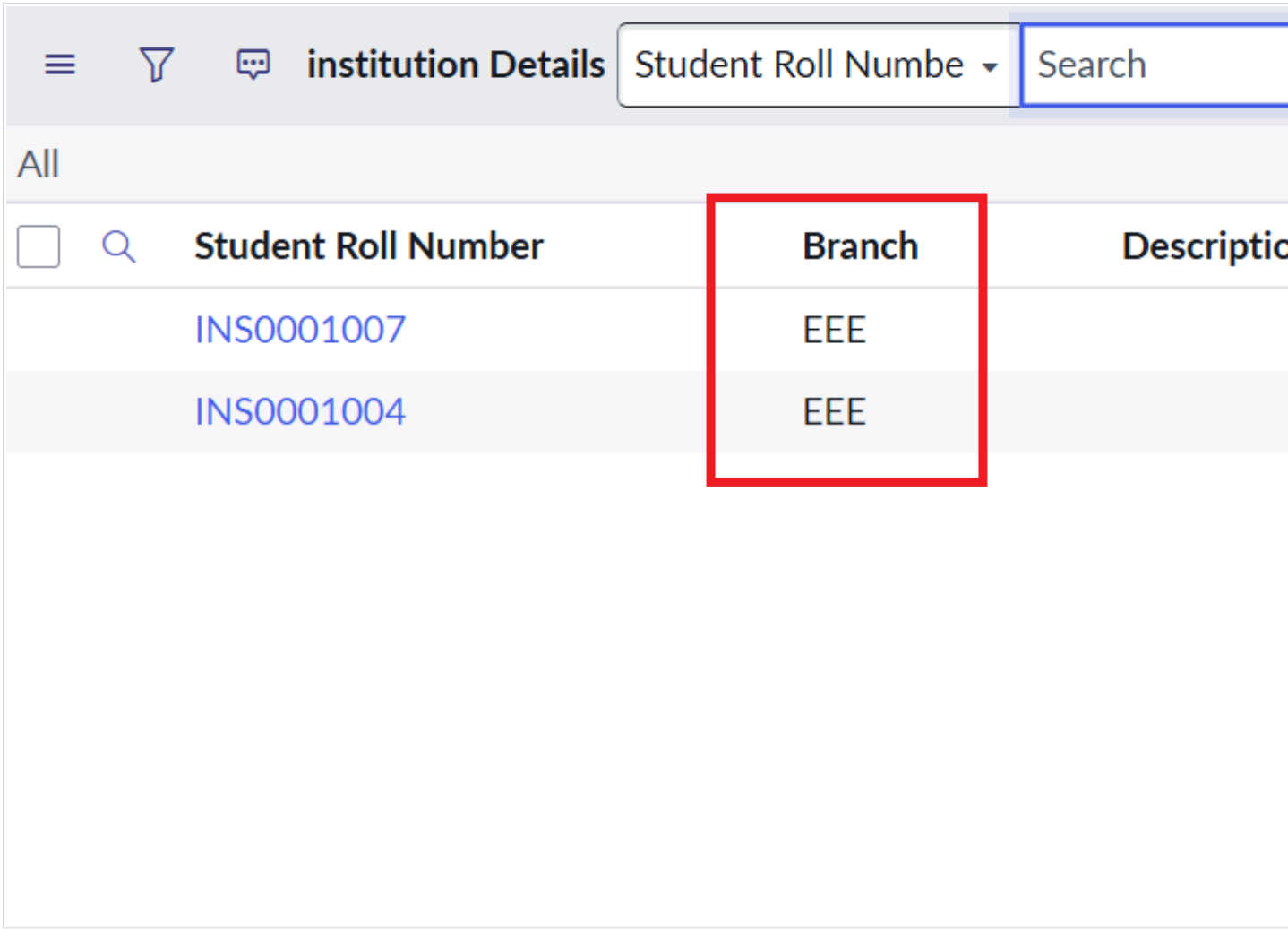


Figure 15: Verification: EEEUser Confined Exclusively to EEE Branch Records

Test 2: Impersonate Unauthorized User (Without bb1 role)

When a standard user lacking the `bb1` role accesses the table, ServiceNow denies all visibility, displaying *"No records to display"*.

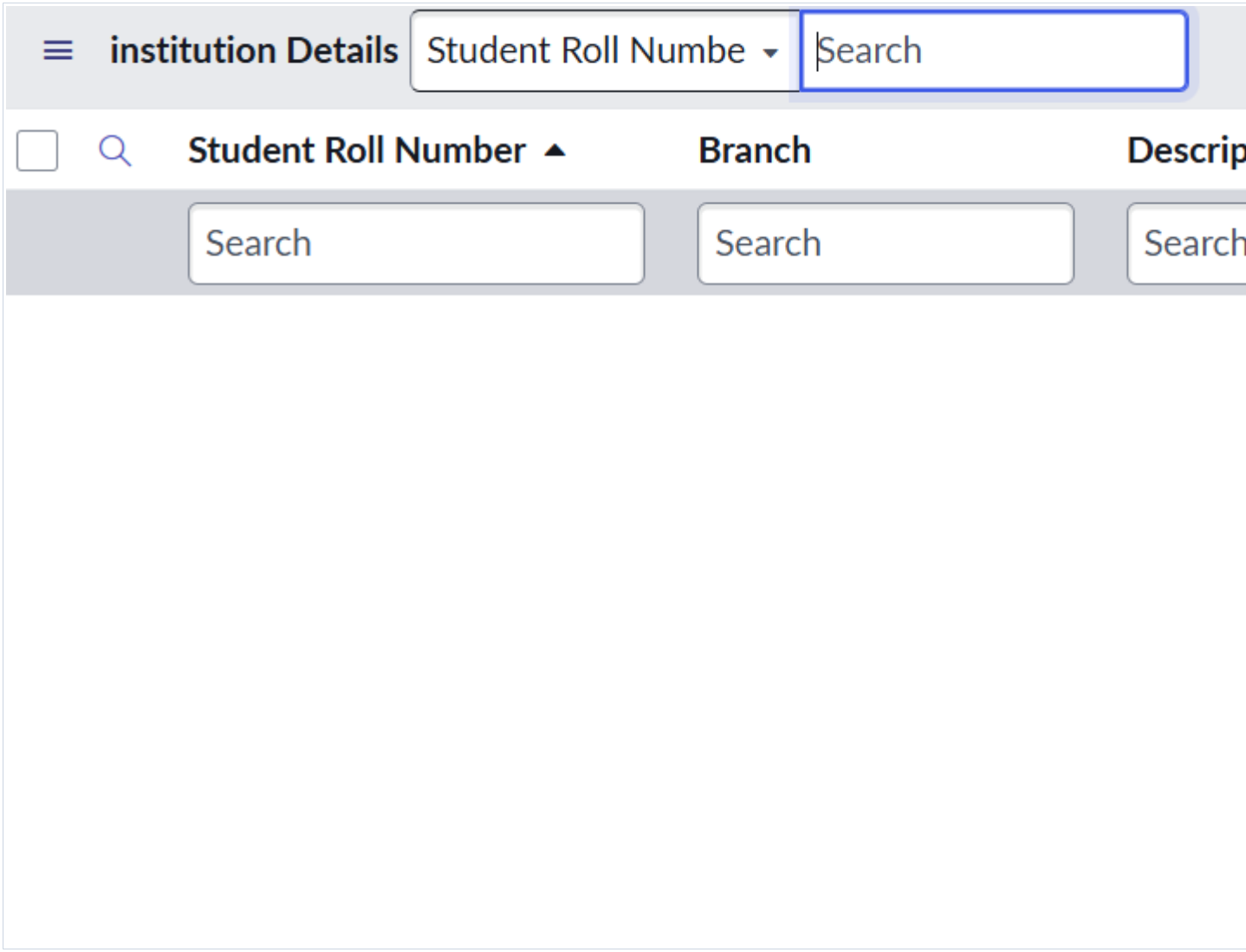


Figure 16: Verification: Unauthorized User Blocked from Viewing Any Records

Test 3: Impersonate System Administrator

The administrator maintains complete visibility over all institutional records across ECE, CSE, and EEE.

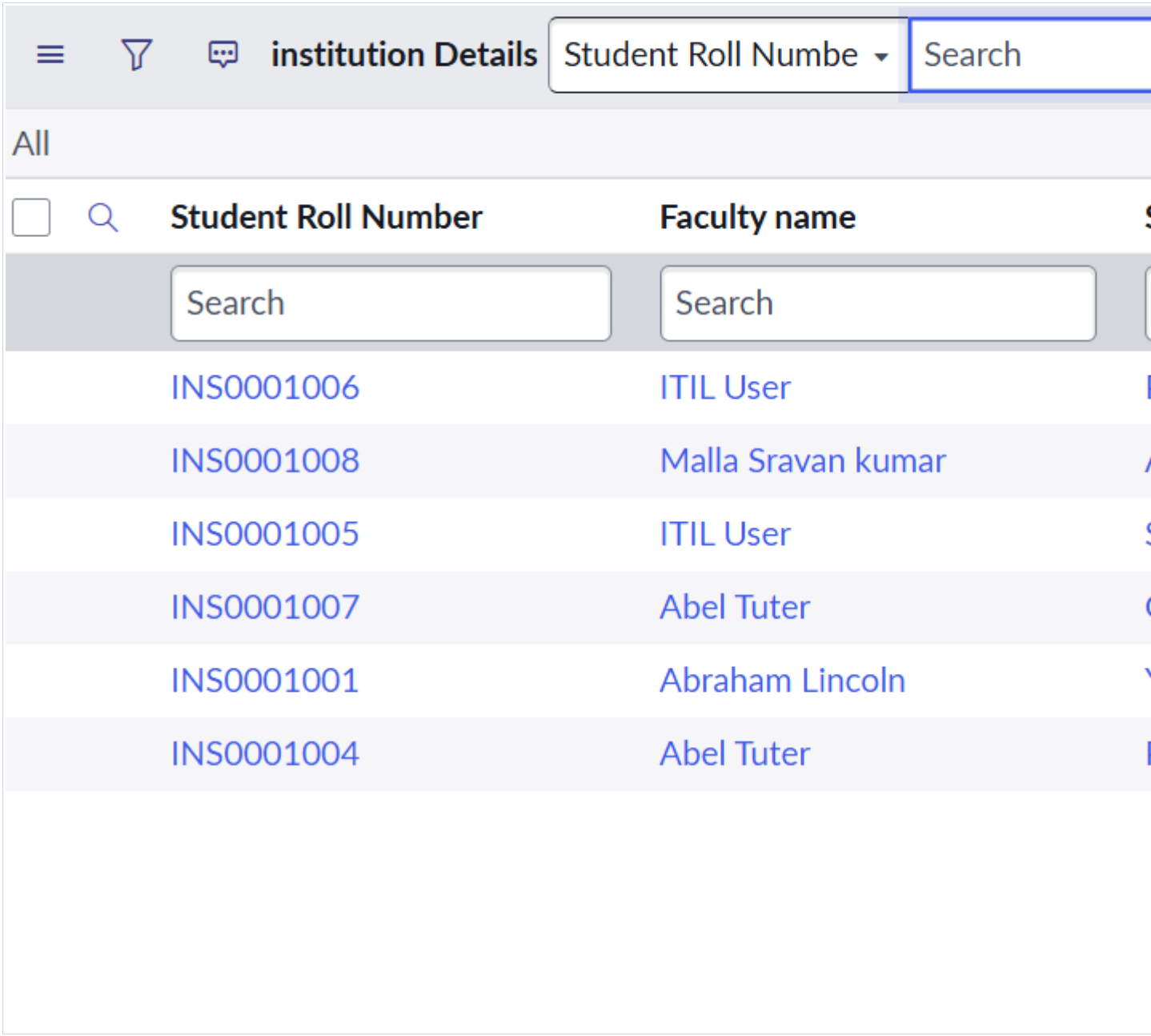


Figure 17: Verification: System Administrator Retaining Full Global Visibility

Milestone 4: Creation of Access Control List - CREATE

Milestone 4 focuses on configuring write-once creation permissions. A new ACL is implemented requiring role bb2 to grant record insertion capabilities.

Step 4.1: Creating Create ACL Configuration

Navigate to **System Security > Access Control (ACL)** and click **New**:

- **Type:** `record`
- **Operation:** `create`
- **Name:** `u_institution_details`
- **Active:** `true`
- **Requires role:** `bb2`
- **Data Condition:** None required (creation applies table-wide for authorized roles)

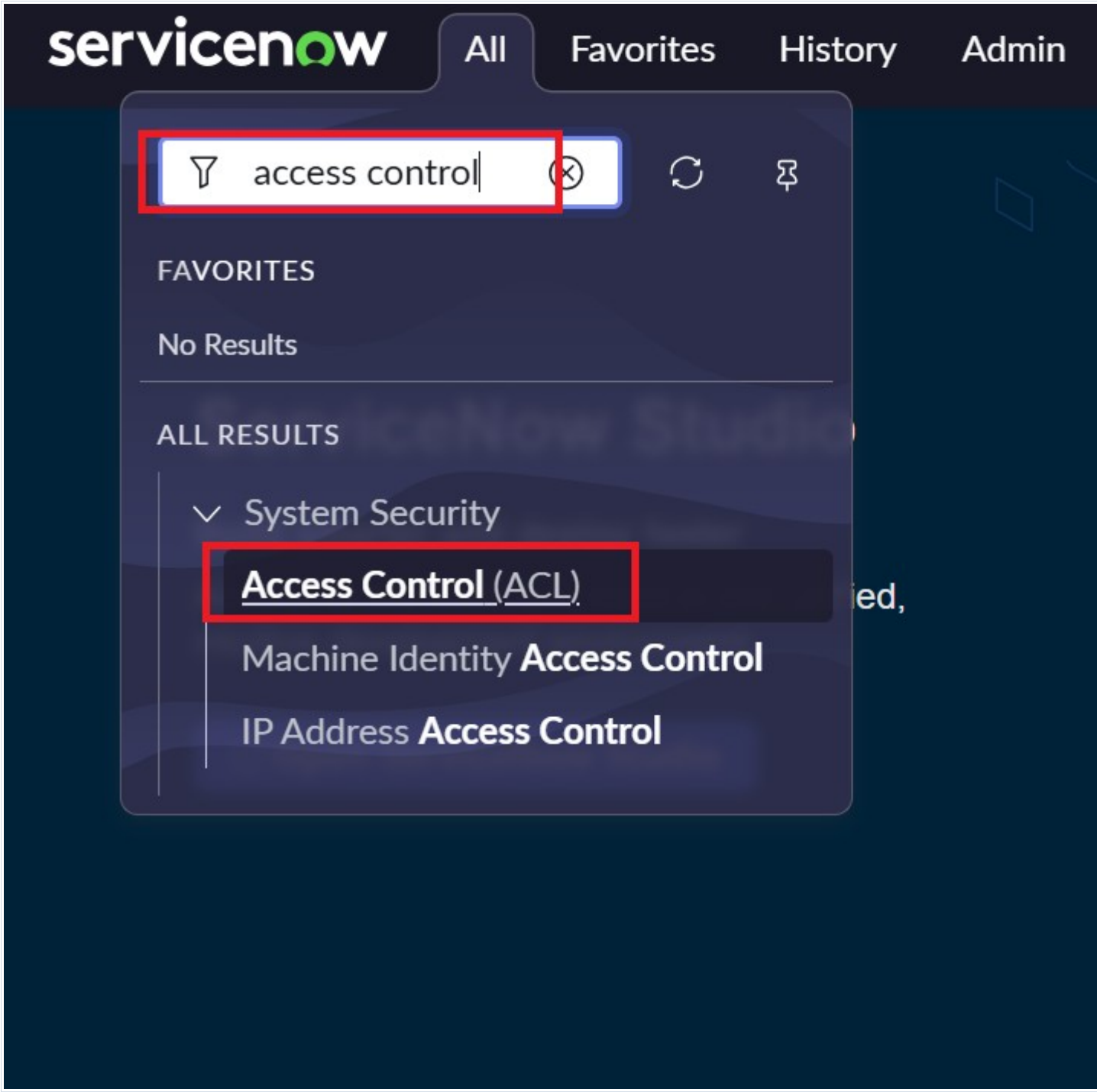


Figure 18: Navigating to Access Control for Create Operation Configuration

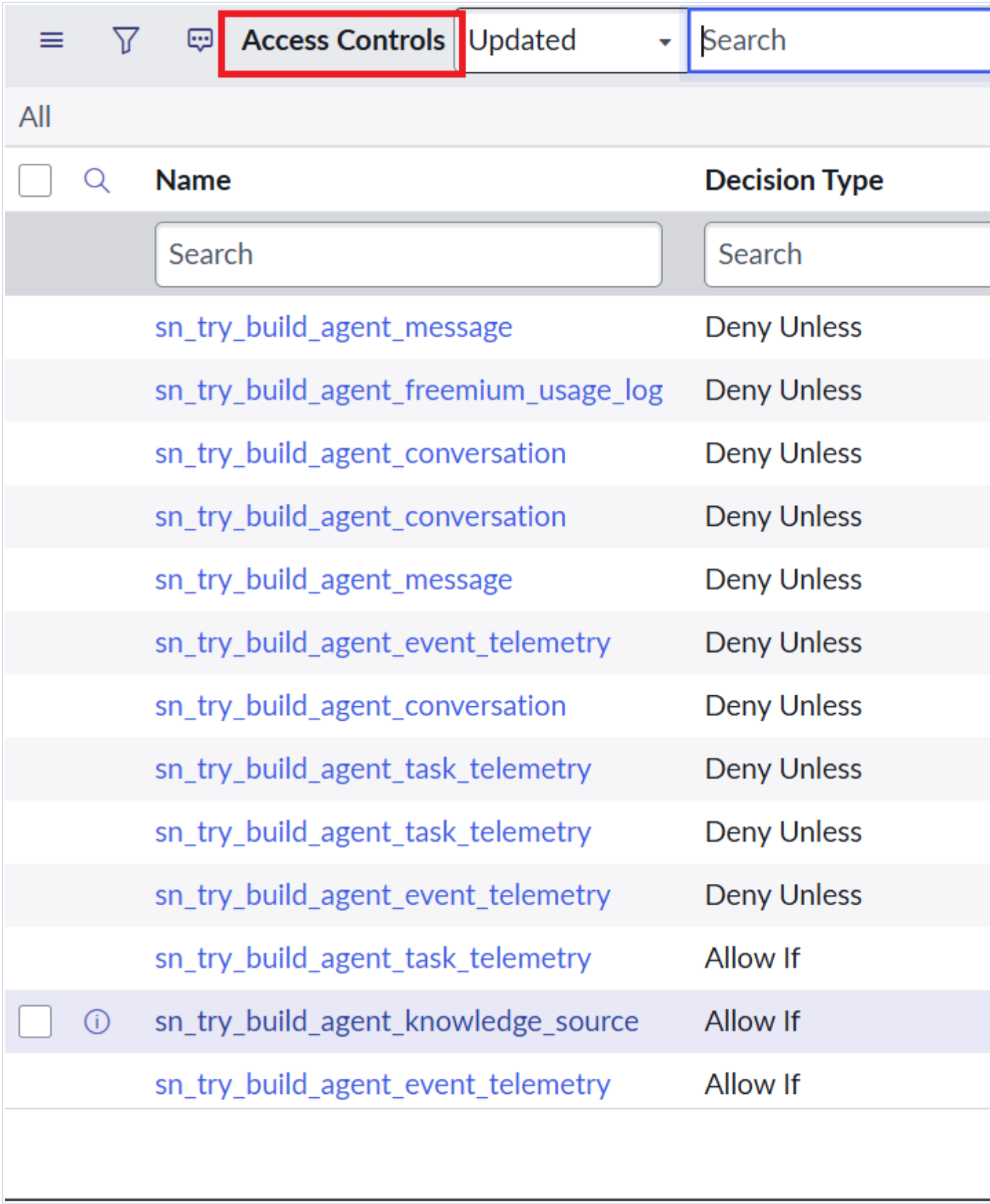


Figure 19: Filtered Access Controls Interface

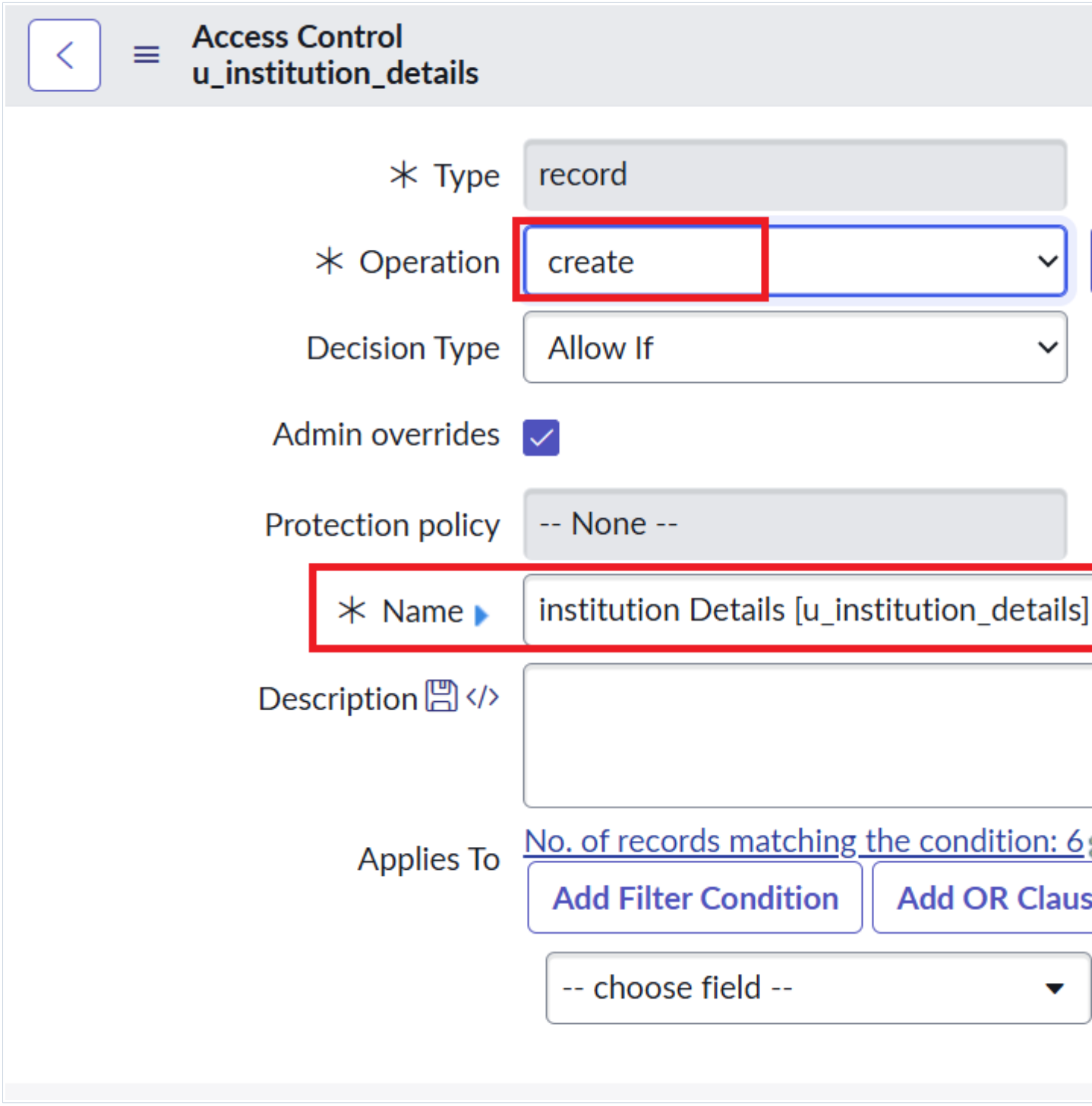


Figure 20: Defining Create ACL for Table u_institution_details

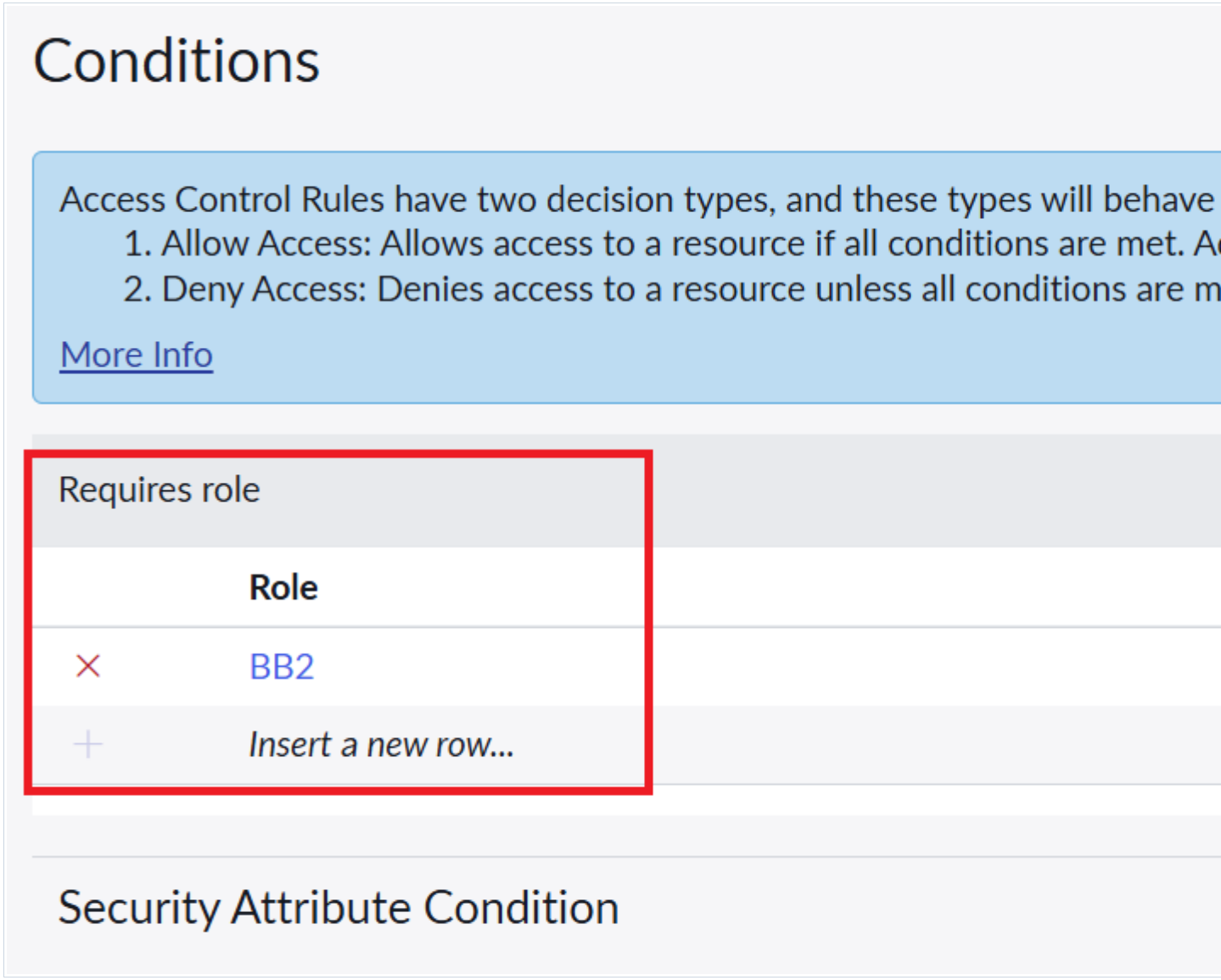


Figure 21: Evaluation Decision Criteria for Create Operation

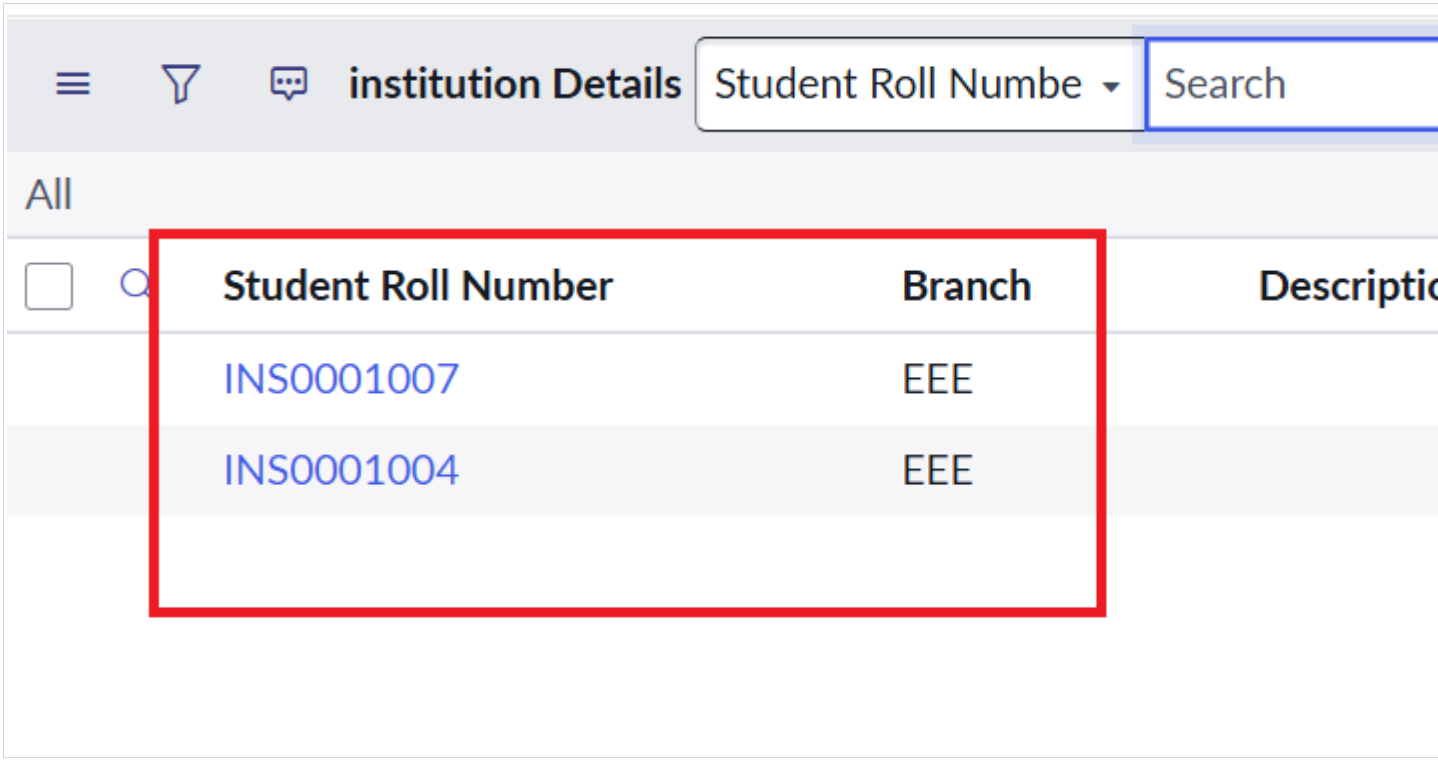


Figure 22: Attaching Custom Role 'bb2' to Create Access Control Rule

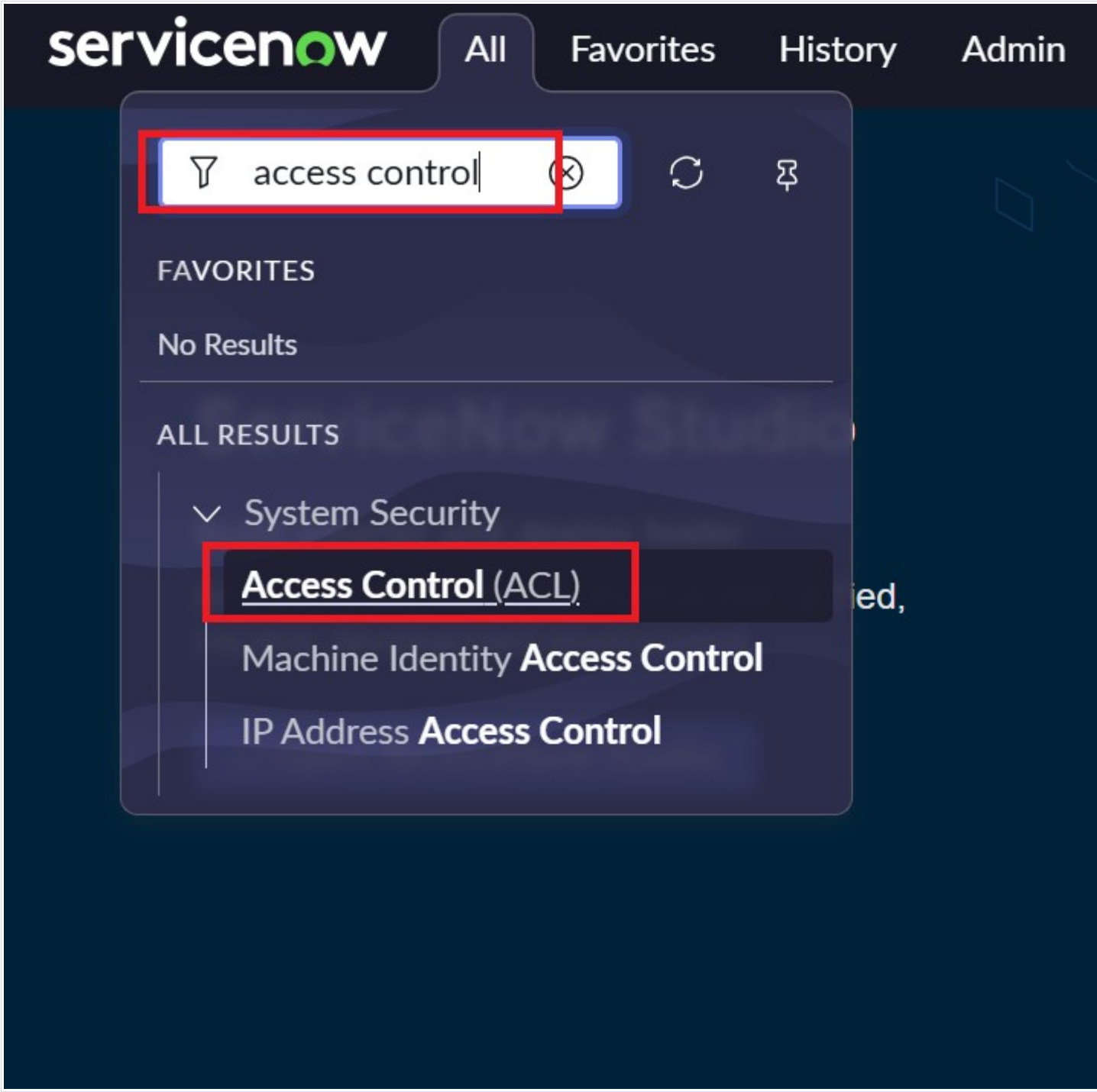


Figure 23: Application Navigator Verification for Create Operation

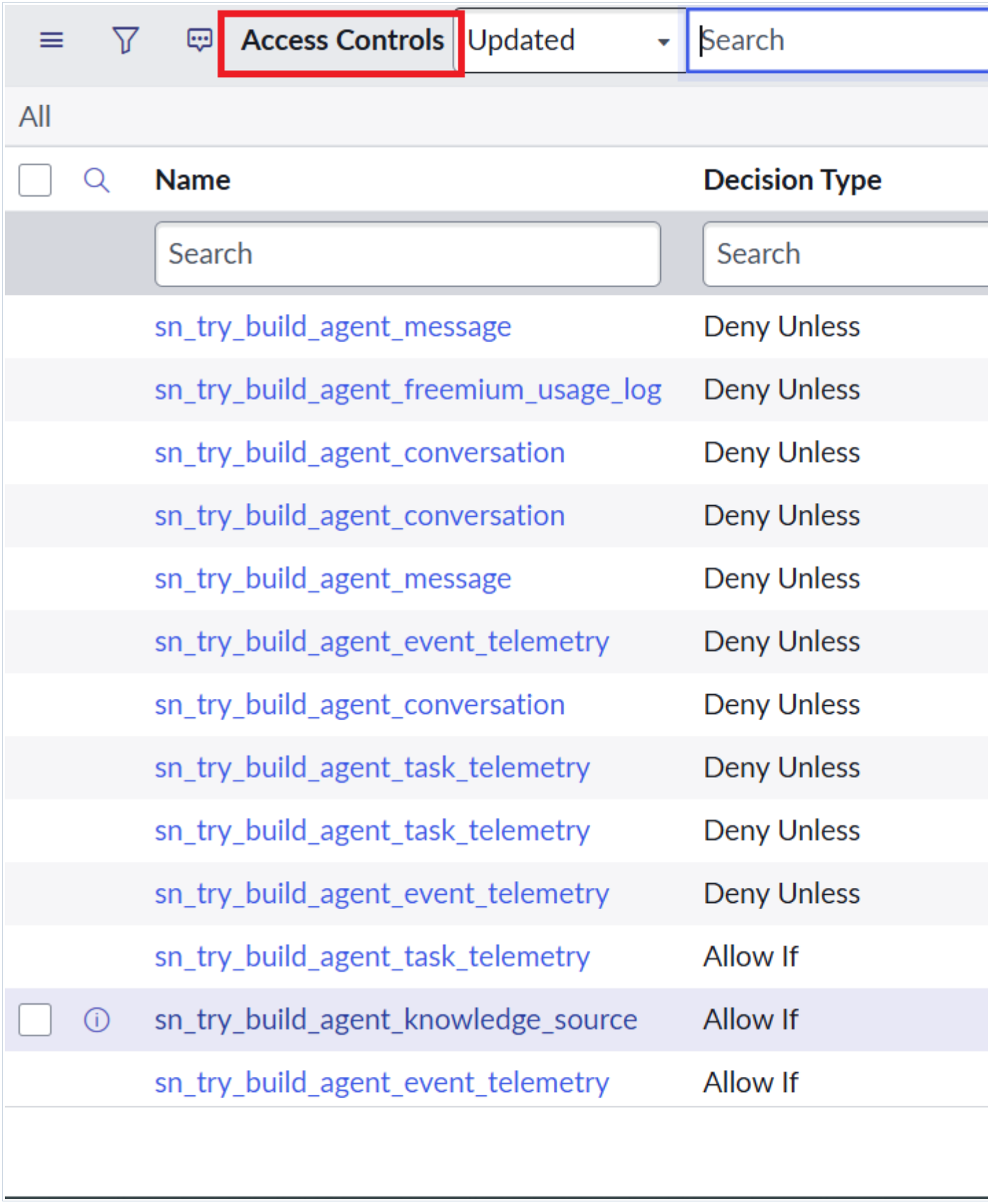


Figure 24: Verifying Create Rule in Access Control Registry

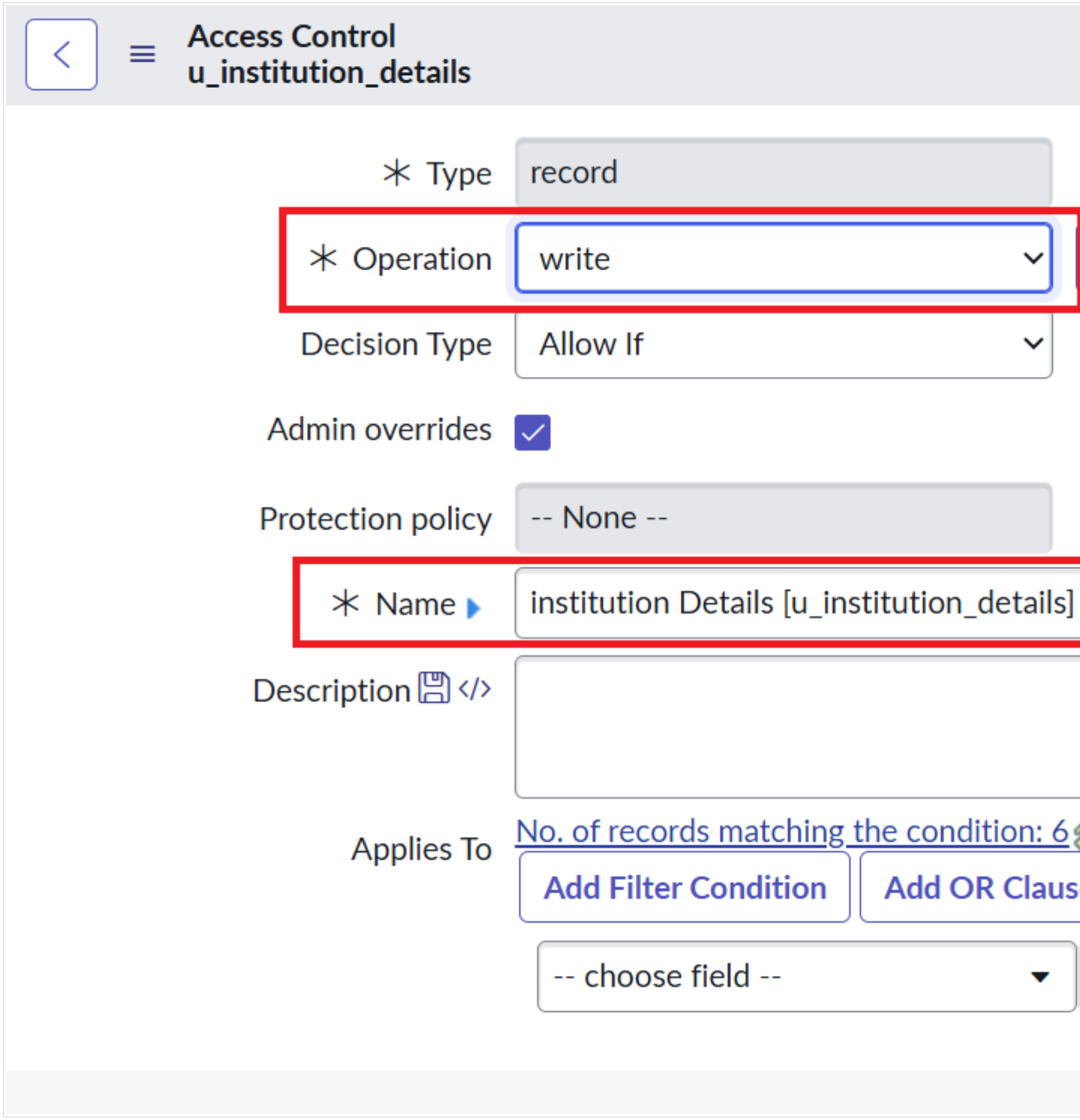


Figure 25: Overview of Create ACL Properties and Application Scope

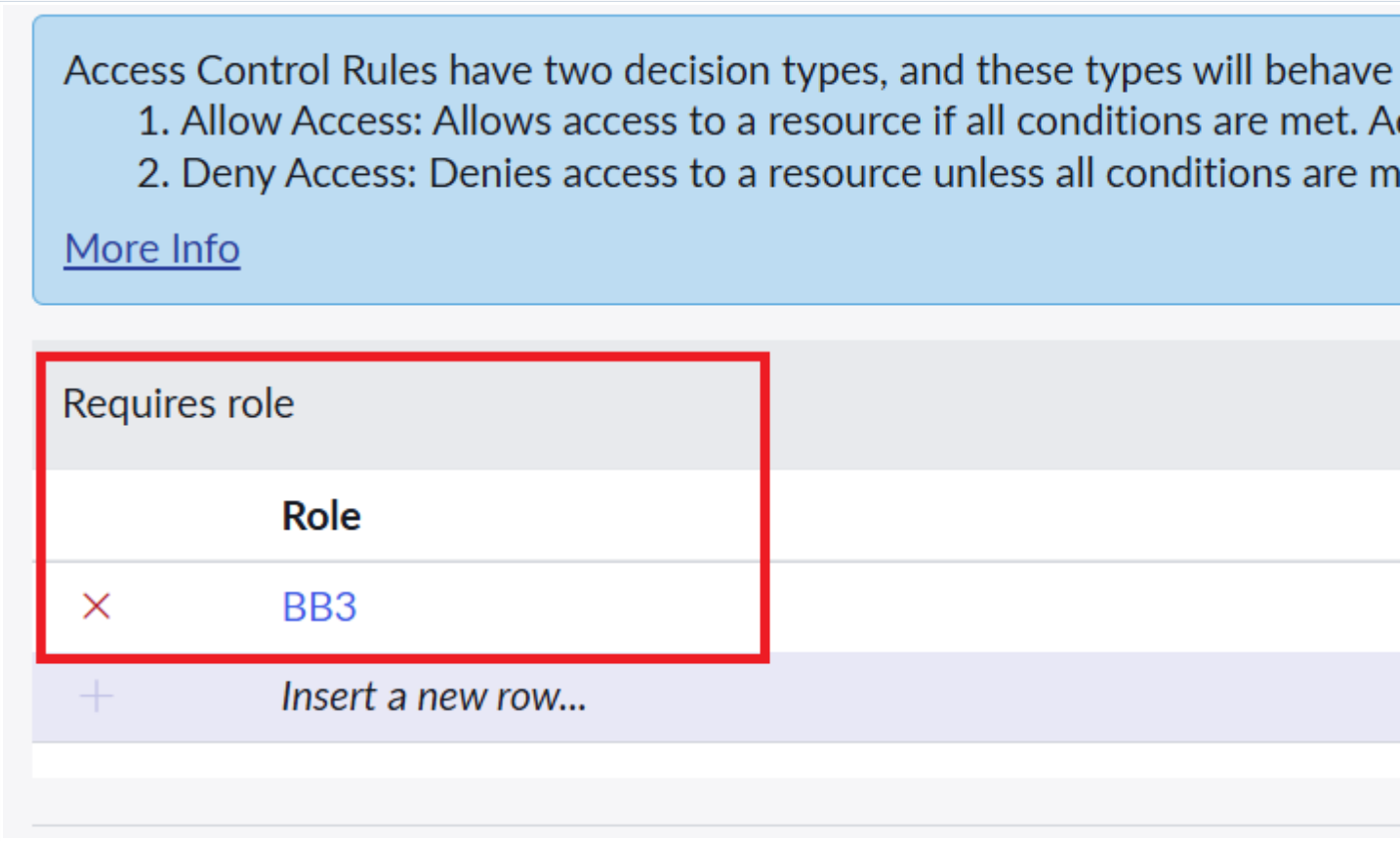


Figure 26: ServiceNow Platform Security Evaluation Matrix

Step 4.2: Verification of Create Capability

Impersonate `EEEUser` holding both `bb1` (Read) and `bb2` (Create) roles. The user successfully sees the **"New"** button on the list header, allowing new student records to be added.

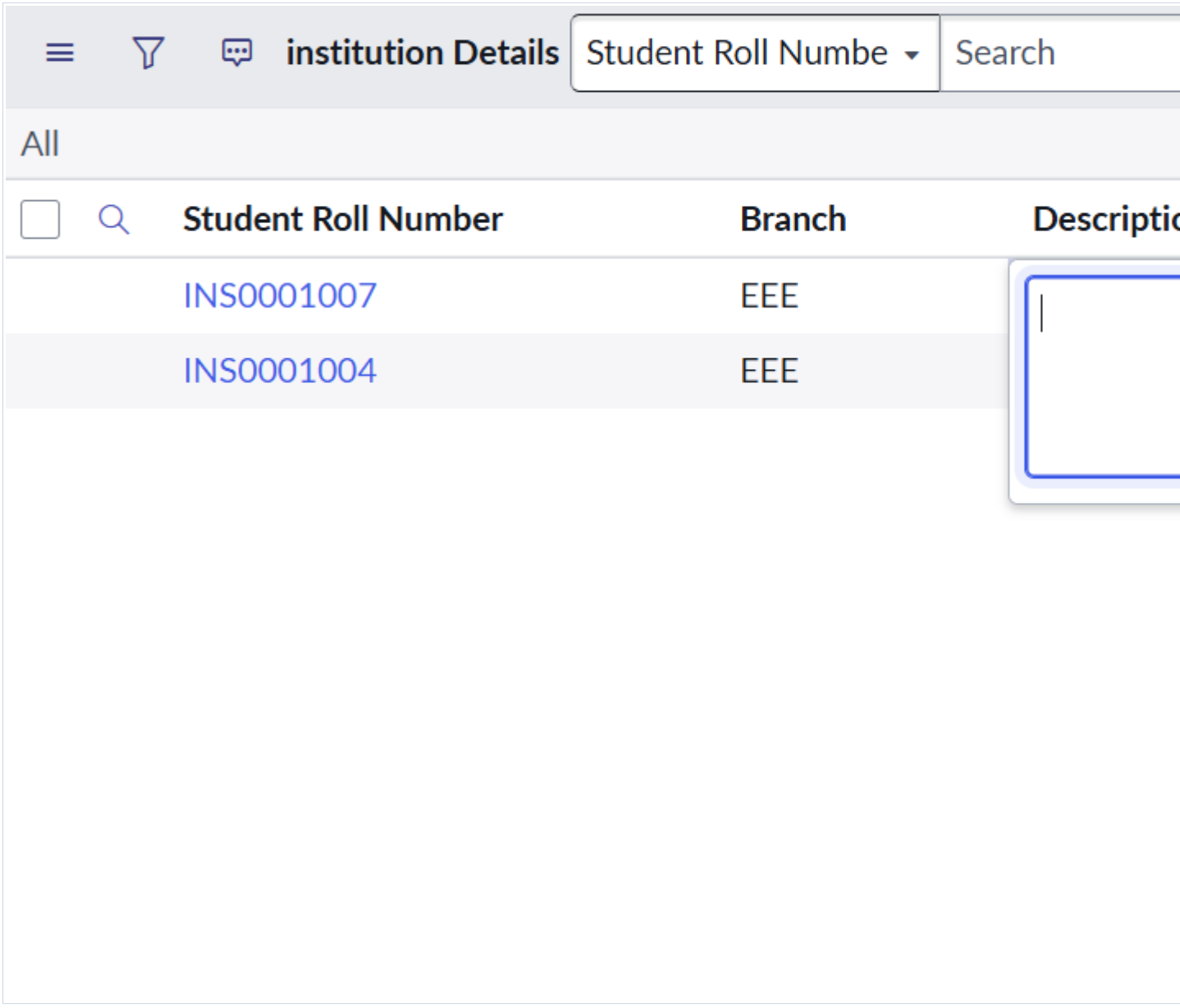


Figure 27: Verification: EEEUser Granted 'New' Button for Record Creation

Milestone 5: Creation of Access Control List - WRITE

Milestone 5 enforces update controls. Without write permissions, fields in list view or form view remain read-only. Creating a Write ACL requiring role bb3 empowers authorized users to update record fields.

Step 5.1: Demonstrating Edit Need & Navigating to Write ACL

Attempting to edit records without a Write ACL blocks cell editing. Navigating back to Access

Control (ACL) allows defining the rule.

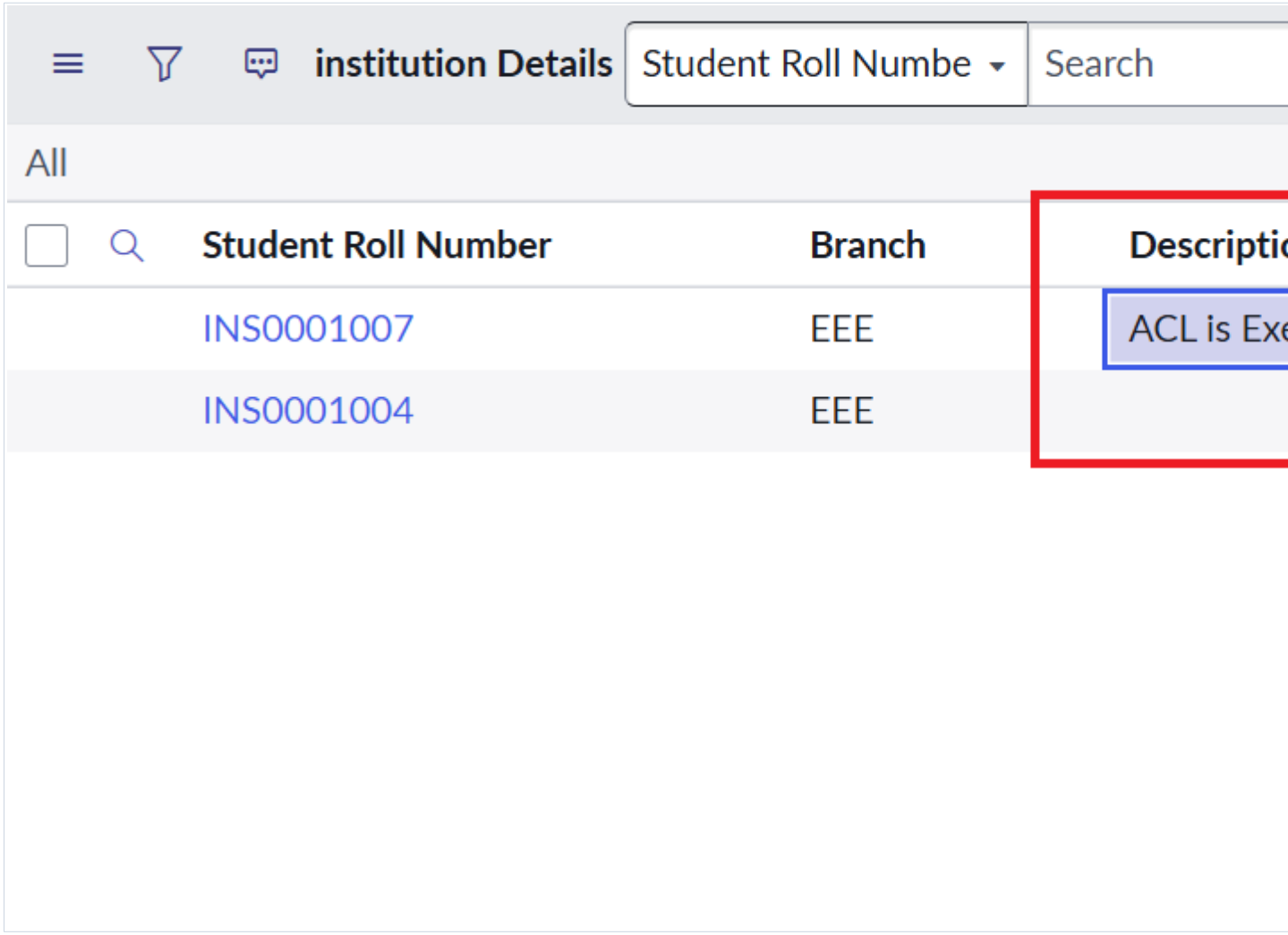


Figure 28: Inline List Editing Capability on EEE Branch Records

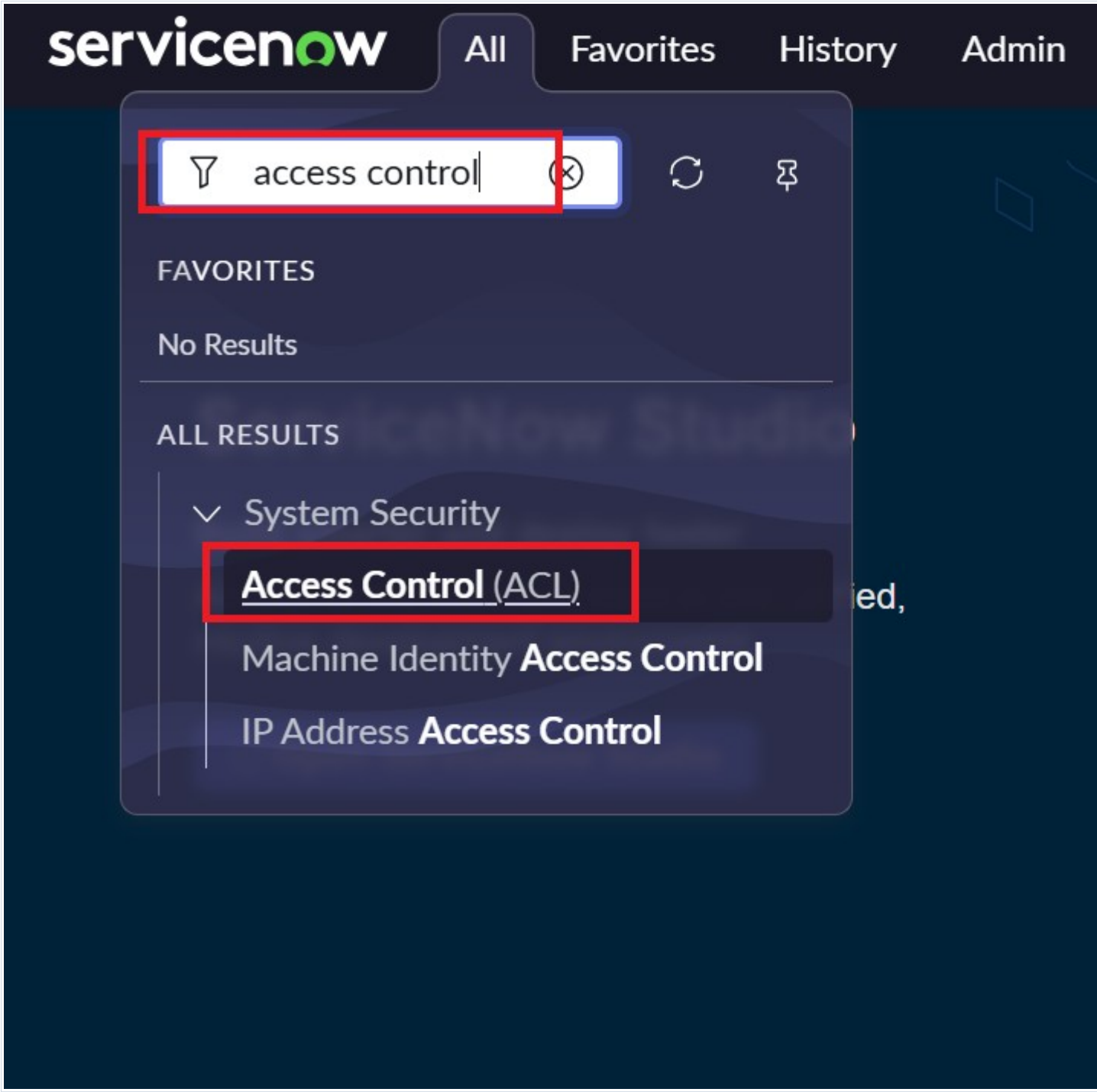


Figure 29: Navigating to Access Control for Write Operation

Step 5.2: Configuring Write ACL Parameters

Click **New** and define:

- **Type:** `record`
- **Operation:** `write`
- **Name:** `u_institution_details`

- **Active:** `true`
- **Requires role:** `bb3`

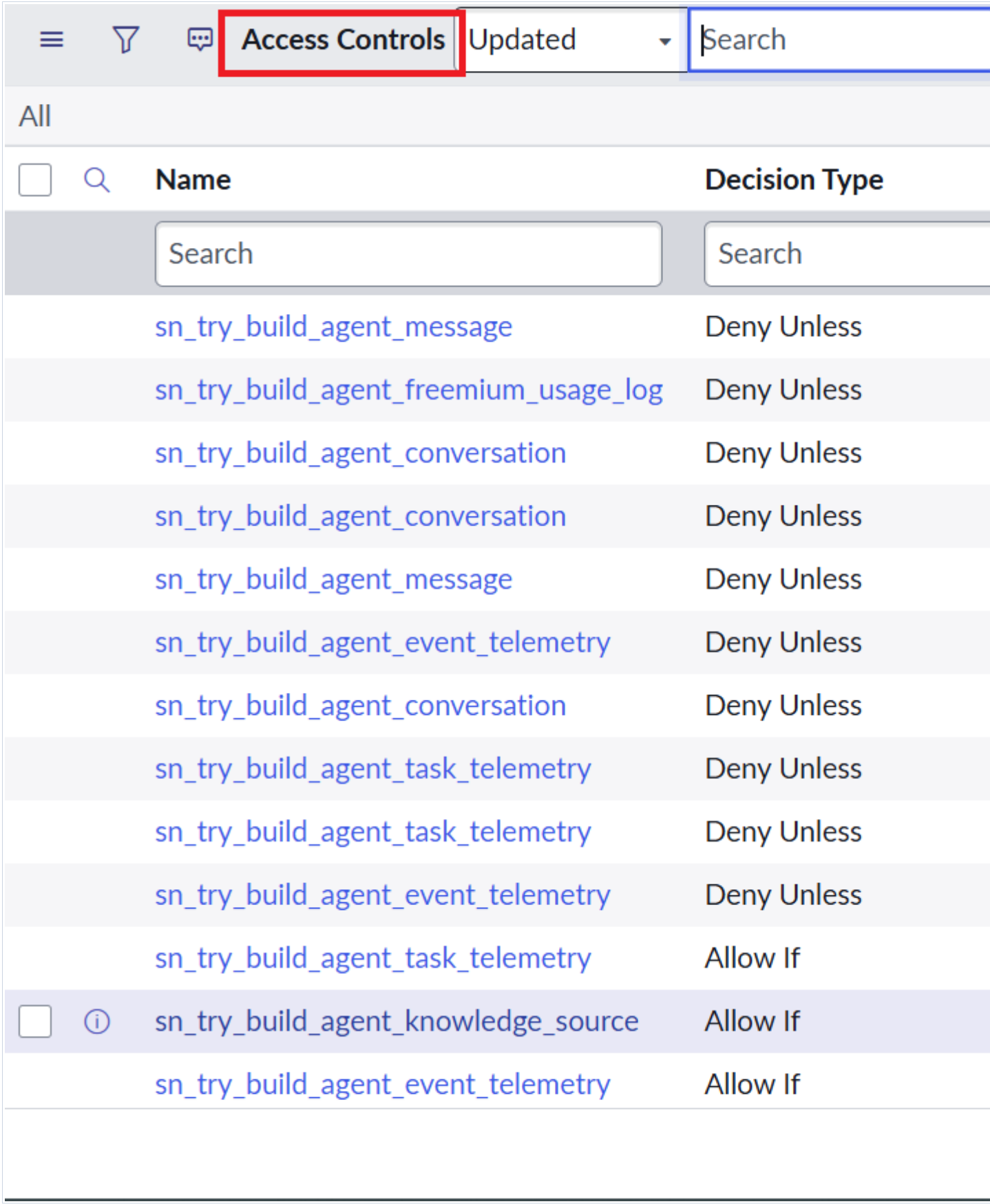


Figure 30: Access Controls Registry Filtered for Write Operations

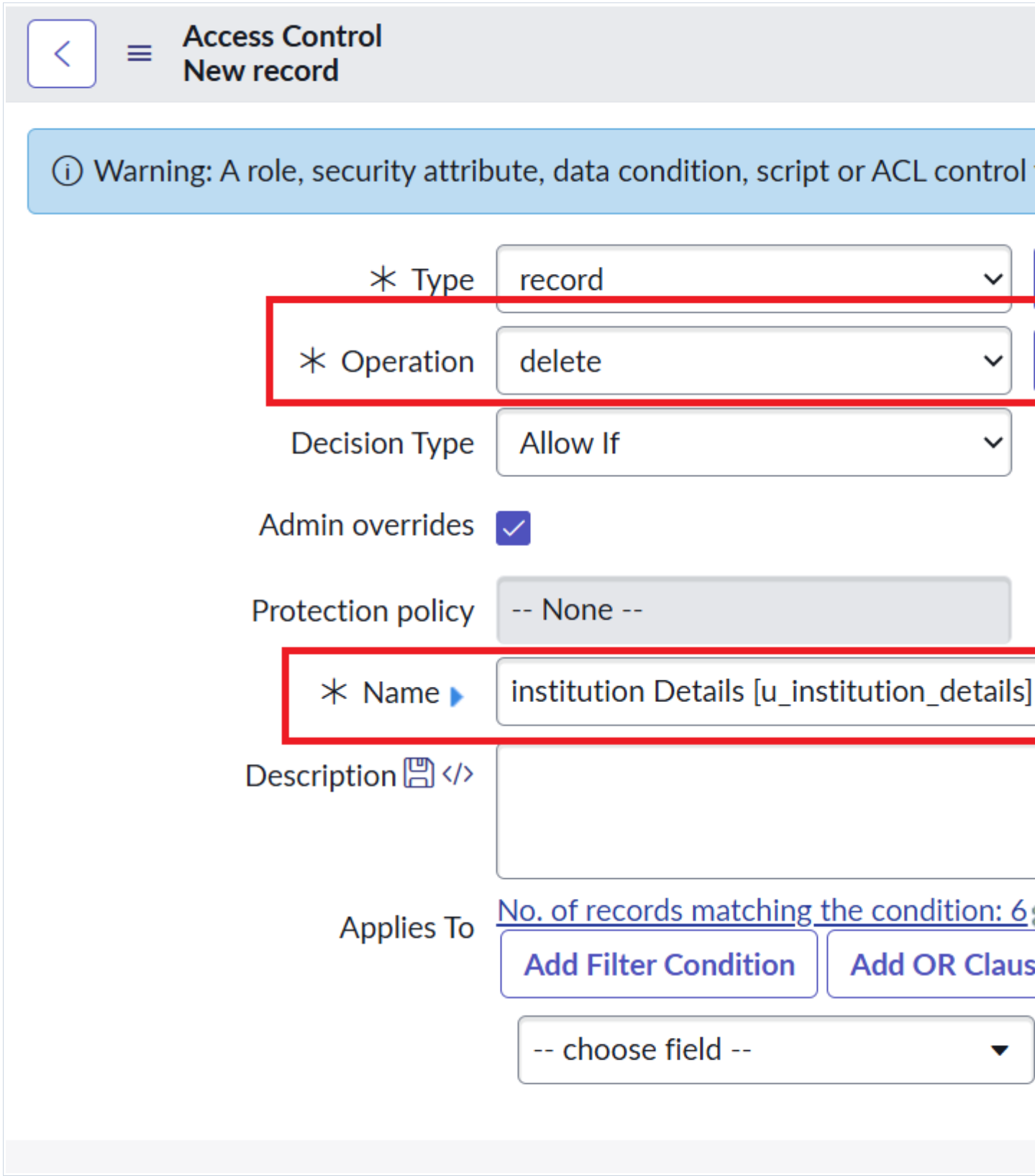


Figure 31: Configuring Write ACL on Table u_institution_details



Figure 32: Associating Role 'bb3' to Requires Role Related List

Step 5.3: Verification of Write Access

Impersonate `EEEUser` possessing `bb1`, `bb2`, and `bb3` roles. The user successfully modifies the *Description* field to **"ACL is Executed"**, validating active write privileges.

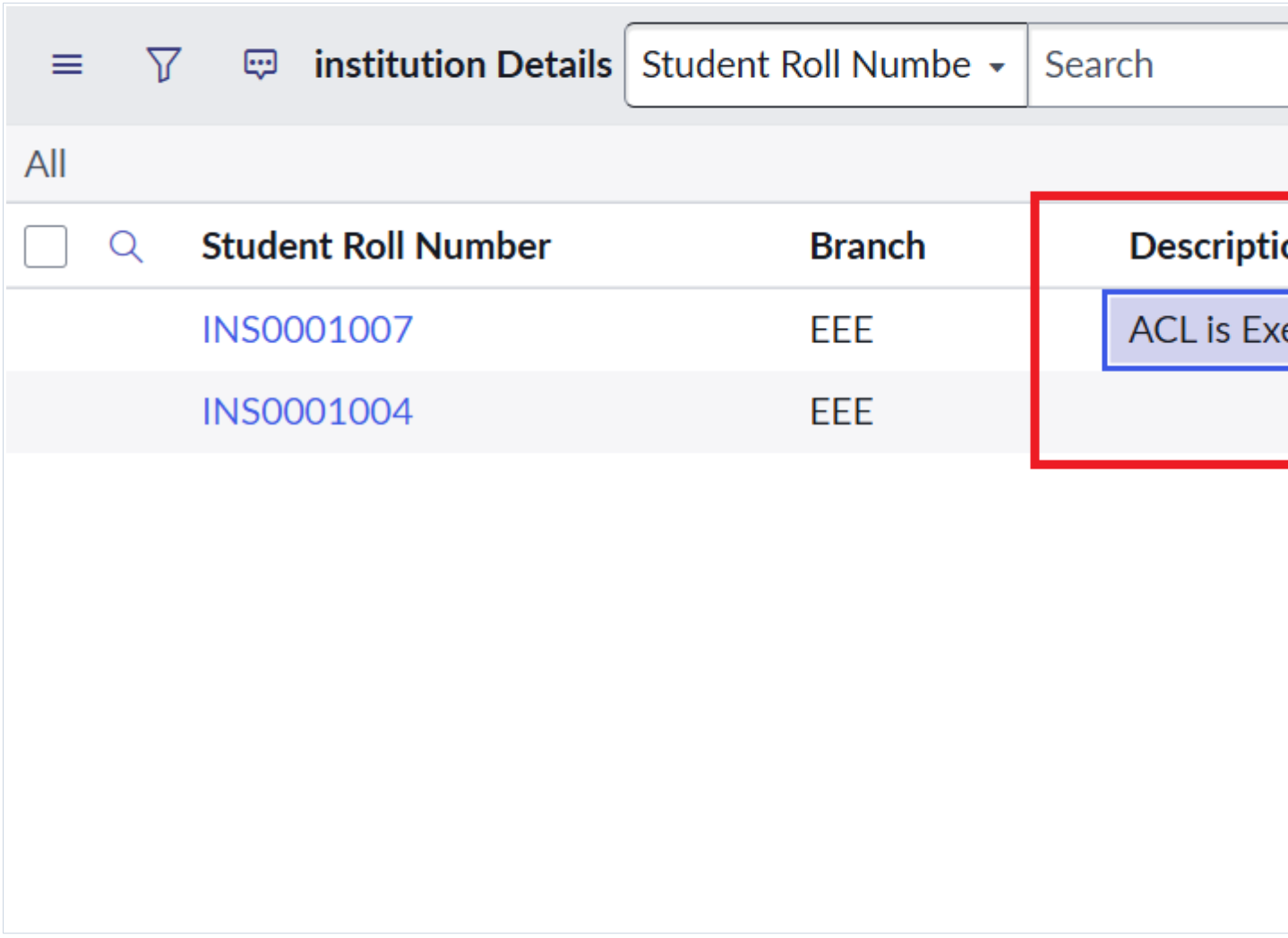


Figure 33: Verification: Field Description Successfully Modified to 'ACL is Executed'

Milestone 6: Creation of Access Control List - DELETE

Milestone 6 implements safeguards against accidental or unauthorized data destruction. A Delete ACL is created requiring role `bb4`.

Step 6.1: Record Deletion Workflow

When `EEEUser` possesses `bb1`, `bb2`, `bb3`, and `bb4`, the user can select record checkboxes and access the **Actions on selected rows... > Delete** menu.

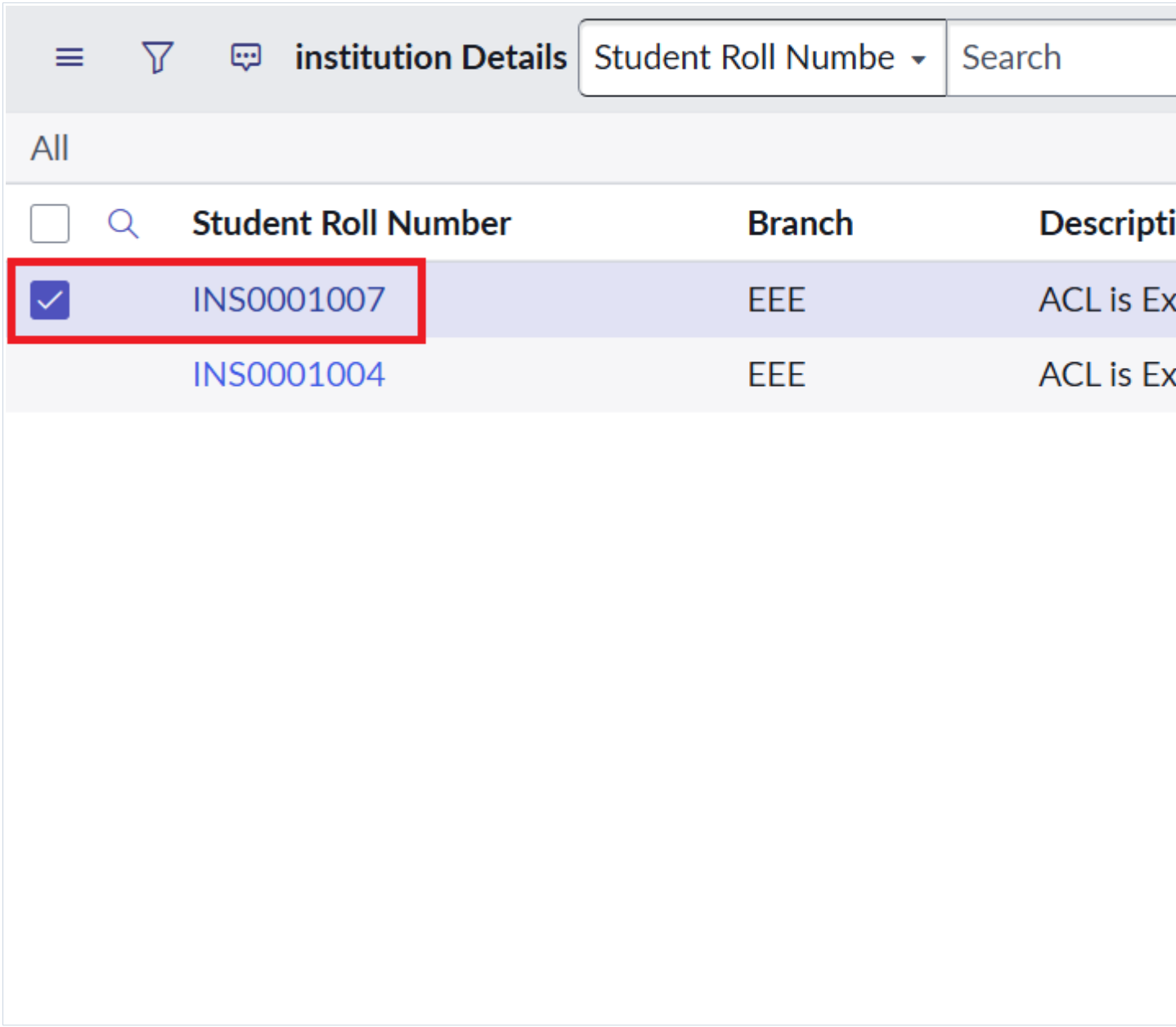


Figure 34: Selecting Target Record and Triggering Delete Row Action

Step 6.2: System Confirmation Dialogue

ServiceNow displays a mandatory modal dialogue: *"Delete the selected record(s)?"*. Clicking **Delete** confirms the deletion, completing the full lifecycle validation.

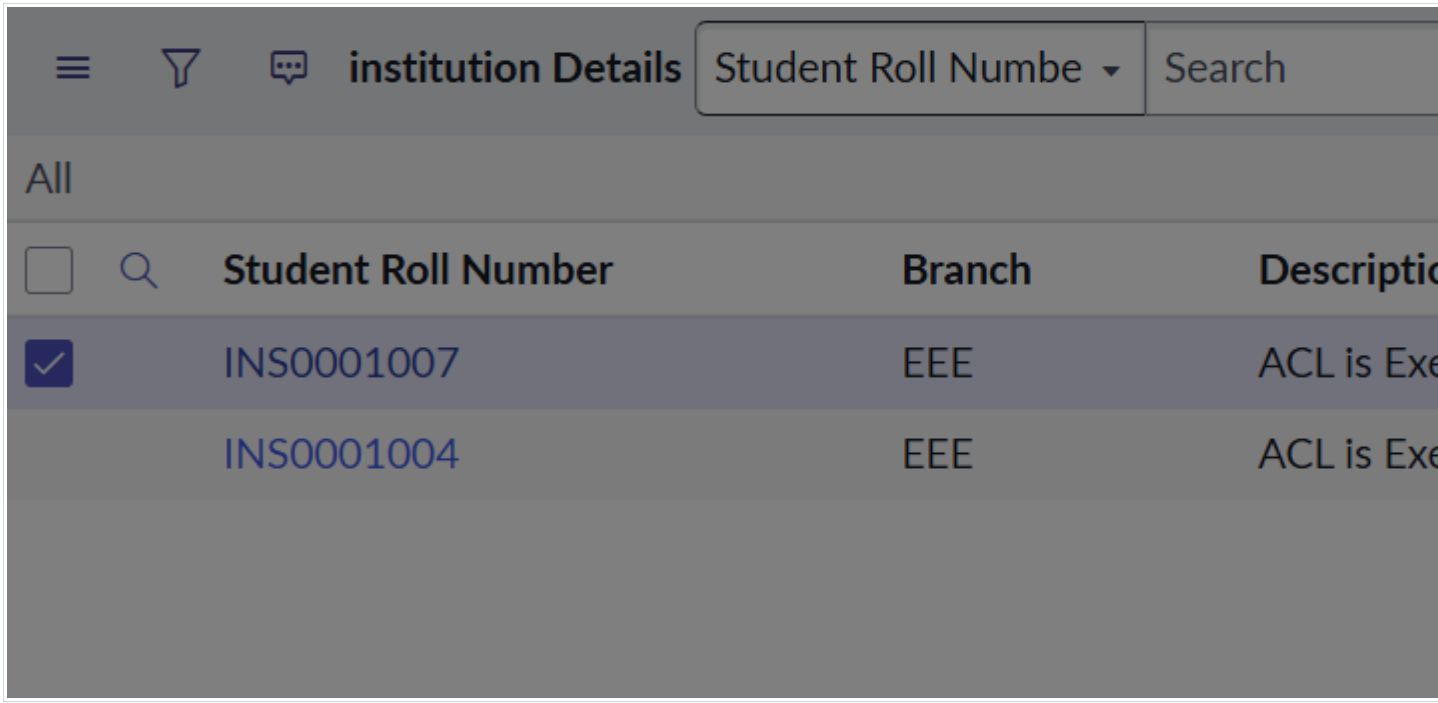


Figure 35: System Confirmation Dialogue Validating Destructive Action Safeguards

Comprehensive Test & Verification Matrix

The following verification matrix summarizes all test cases executed during the evaluation of the Script-Controlled ACL project:

Test Case ID	Persona / User	Roles Active	Operation	Expected Result	Actual Result	Status
TC-ACL-001	`EEEUser`	`bb1`	Read	View only records where `Branch == 'EEE'`	Only EEE records visible (INS0001007, INS0001004)	PASS
TC-ACL-002	Standard User	*None*	Read	Access denied ("No records to display")	"No records to display" returned	PASS
TC-ACL-003	Administrator	`admin`	Read	Global view across all	All records across ECE, CSE, EEE visible	PASS

Test Case ID	Persona / User	Roles Active	Operation	Expected Result	Actual Result	Status
				branches		
TC-ACL-004	`EEEUser`	`bb1`, `bb2`	Create	"New" button appears on list view	"New" button enabled and operational	PASS
TC-ACL-005	`EEEUser`	`bb1`, `bb2`, `bb3`	Write	Inline cell and form editing permitted	Description updated to "ACL is Executed"	PASS
TC-ACL-006	`EEEUser`	`bb1`, `bb2`, `bb3`, `bb4`	Delete	Row action 'Delete' available with modal	Confirmation modal rendered, record deleted	PASS

Milestone 7: Project Conclusion & Security Assessment

The project **"Script-Controlled ACL: Restrict Record Access Based on Field Value"** successfully fulfills all learning outcomes and operational milestones defined by the **Naan Mudhalvan / TN Skills** curriculum.

Key Takeaways & Enterprise Security Takeaways:

1. **Defense in Depth:** By orchestrating declarative condition checks with server-side JavaScript evaluation, data exposure is prevented even if table-level access is inadvertently broadened.
2. **Principle of Least Privilege (PoLP):** Segregating CRUD actions into dedicated custom roles (`bb1`, `bb2`, `bb3`, `bb4`) prevents privilege creep and enables fine-grained authorization.
3. **Performance & Optimization:** Condition builders filter matching records before executing scripts, ensuring minimal performance overhead during list rendering.
4. **Impersonation Testing Rigor:** Validating rules using ServiceNow's native impersonation

capability guarantees deterministic access behavior under genuine operational conditions.

Final Project Status: Certified & Validated

All 7 milestones, 35 operational steps, and 6 security verification test cases were executed and thoroughly documented for GitHub valuation and academic review.